ngspice-46 — Full Change Report

Every modification and its reason (original → current)

Ngspice + OpenVAF Enhancements project

July 2026

Contents

ngspice-46 — Full Change Report 3
1. The OSDI ABI, version 0.4 $\rightarrow$ 0.7 . . . 3
2. The OSDI runtime — `src/osdi/` . . . 4
3. Analyses — `src/spicelib/analysis/` . . . 5
4. Frontend — `src/frontend/` . . . 7
5. Parser and device registry — `src/spicelib/` . . . 8
6. Maths — `src/maths/` . . . 8
7. Build system . . . 10
8. Per-enhancement index . . . 10

ngspice-46 — Full Change Report

Every modification applied to ngspice-46 in this project, with its reason. The baseline is the pristine ngspice-46 source tree (as vendored in the project's `original/` snapshot, which already contains OpenVAF-Reloaded's stock OSDI support); the current state is the tree committed in this repository under `ngspice-46/`. Each entry links the enhancement write-up in `enhancements_doc/` that carries the full engineering detail and the verifying example suite. The companion document openvaf_changes_full-report.md covers the compiler side.

> Maintenance note: this report is updated whenever an enhancement touches ngspice sources. If you are reading it alongside a newer tree, the per-enhancement index at the end tells you the last change it covers.

Scope summary. One new source file and 41 modified ones carry all the functional changes (~2,000 substantive diff lines). A further ~100 `Makefile.in` files and `config.h.in` differ only because the autotools were regenerated with a current libtool (E-77); they contain no hand-written changes and are not listed individually. Everything below is behavior, interface, or diagnostics.

---

1. The OSDI ABI, version 0.4 → 0.7

`src/osdi/osdi.h` is the authoritative C contract between ngspice and the `.osdi` objects `openvaf-r` produces. The project extended it four times — twice additively (new exported symbols old loaders simply never look up), twice with genuine version bumps (layout changes):

Change	Kind	Enhancement	Why
`OSDI_ABSDELAY_COUNT` / `OSDI_ABSDELAY_INFOS` exports	additive symbols	E-1	`absdelay()` needs simulator-side waveform history and synthetic delay-equation rows; the descriptors tell ngspice which nodes/offsets each delay slot uses
`OSDI_LAST_CROSSING_COUNT` / `OSDI_LAST_CROSSING_INFOS` exports	additive symbols	E-6	`last_crossing()` needs waveform history and crossing interpolation — same additive pattern as absdelay
`OsdiNode.nodeset` field	ABI 0.5 (node stride change)	E-45	Verilog-A net initializers (`electrical a = 5.0;`) become solver initial-guess nodesets; every node record carries the hint
`num_ac_stim_src` / `ac_stim_sources` / `load_ac_stim` descriptor tail	ABI 0.6 (descriptor grows)	E-51	full `ac_stim()` support: a Verilog-A module can *be* the AC stimulus, so the descriptor must enumerate its AC right-hand-side sources
`load_noise` destination stride 1 → 2 (signed `(flat, react)` power pairs)	ABI 0.7	E-54	correct noise physics: operating-point-dependent factors and `ddt()`-shaped (jω) noise need a complex amplitude per source, `re + jω·im`, so coherent same-named sources (LRM 4.6.4) sum with correct sign and frequency shape

Two additive flag conventions ride on the existing `eval()` interface (not version bumps):

- `EVAL_FLAG_IS_FINAL_STEP` (bit 1≪21) in the eval *input* flags — `@(final_step)` firing (E-53);
- eval *return* flags for `$finish`/`$stop` requests and `$discontinuity`-driven step rejection (E-55).

Pairing consequence: this ngspice requires OSDI $\geq$ 0.7 and rejects older objects with a clear version message; `.osdi` files from older compilers must be recompiled (E-54).

---

2. The OSDI runtime — `src/osdi/`

`osdiaccept.c` — new file (E-55, E-1, E-6)

The accepted-timepoint hook (`DEVaccept`) OSDI previously lacked. It advances the `absdelay/last_crossing` waveform histories only at *accepted* points (so rejected timesteps don't corrupt the delay lines) and reads the per-attempt latched `$finish`/`$stop` request flags at the one boundary where honoring them is safe. Why: acting on `$stop` mid-Newton-iteration was treated as a step failure and ground the timestep into a rejection loop; `$finish` was ignored outright.

`osdidefs.h` (104 diff lines)

- `OsdiExtraInstData` grows the per-instance fields behind every runtime feature: `dt/temp` offsets with given-flags (instance temperature), `eval_flags`, absdelay waveform-history rows and pre-allocated KLU/SPARSE matrix-pointer sets for the delay-equation rows (re-pointed on every DC$\leftrightarrow$AC transition, mirroring the regular Jacobian handling), and the last_crossing history (E-1, E-6, E-55).
- `ALIGN` macro renamed `OSDI_ALIGN`: the macOS SDK's `<sys/param.h>` defines an unrelated `ALIGN`, producing a redefinition warning in every OSDI translation unit (E-77).

`osdiitf.h` / **`osdiext.h`**

The registry-entry structure gains the absdelay/last_crossing descriptor pointers, and `OSDIpendingRequests()` is exported so the analyses can ask "did any device request `$finish`/`$stop` at this accepted point?" (E-1, E-6, E-55).

`osdiregistry.c` (68 diff lines)

- Loads the absdelay/last_crossing descriptor arrays from each `.osdi` and threads them into the registry entries (E-1, E-6).
- Requires OSDI $\geq$ 0.7 (E-54).

`osdiinit.c` (8 diff lines)

- Registers the `DEVaccept` hook (E-55).
- Publishes the synthetic instance-temperature parameter under both spellings, `dt` and the conventional `dtemp` every built-in device uses; `n1 a b mod dtemp=10` used to fail with *"unknown parameter"* while the underlying plumbing was exact (E-80).

`osdiload.c` (441 diff lines — the largest OSDI change)

- Stamps the absdelay synthetic rows (`y_synth`, `z`) for DC, AC and transient, driving the delay from the recorded history (E-1).
- Evaluates `last_crossing` from the recorded waveform with linear interpolation between the bracketing timepoints (E-6).
- Passes the final-step flag through to `eval()` (E-53).
- Latches eval-return flags per timepoint attempt (`point_eval_flags`, OR-ed across Newton iterations, reset per attempt) so `$finish`/`$stop` under solution-dependent conditions survive to the accepted-point boundary (E-55).

- Folds the stride-2 signed noise-power pairs (`fac·|fac|` semantics) when transferring `load_noise` results (E-54, E-42).

`osdisetup.c` (178 diff lines)

- Creates the synthetic delay-equation unknowns and matrix entries for absdelay (E-1).
- Applies the OSDI nodesets (`OsdiNode.nodeset`) as solver initial guesses (E-45).
- A `$fatal`/`$finish` raised during setup (models rejecting their configuration by design — HiSIM's `$port_connected` guards) used to surface as ngspice's baffling *"impossible error — can't occur"*; it now reads *"a Verilog-A device rejected its configuration during setup"* (E-56).
- An internal OSDI node that appears in no Jacobian entry — a net structurally decoupled from the matrix, e.g. an explicit `ground gnd` reference whose branch contribution `V(p,gnd) <+ …` drops the `gnd` column — used to be allocated its own solver row. That all-zero row/column is benign under Sparse (it decouples to `V=0`) but makes the KLU matrix structurally singular, so KLU returned a wrong DC answer (`groundcontrib: v(p)=0` not `1.5`; `hierbranch`: branch currents `0`). Such nodes are now tied to ground (node 0) at setup, matching how OpenVAF already treats them and fixing both solvers (E-116).

`osdiacld.c` (89 diff lines)

- AC loading gains the `ac_stim` right-hand-side injection: the partitioned AC-stimulus source array is loaded through `load_ac_stim` and added to the AC RHS with exact magnitude and phase, `$mfactor` applied linearly (deterministic signals scale with multiplicity) (E-51, E-26).
- Complex (jω-shaped) noise coupling entries participate in the AC matrix (E-54).

`osdinoise.c` (93 diff lines)

- Per-instance grouping of same-named noise sources: coherent summation of complex amplitudes `(a + jω·b)·T` before squaring, per LRM 4.6.4 — same-named sources correlate (including exact anti-phase cancellation), differently-named ones stay independent (E-42).
- Reads the stride-2 signed pairs of ABI 0.7 (E-54).

`osditrunc.c` (34 diff lines)

- Honors `$discontinuity(n)`'s next-step clamp: a negative `bound_step` sentinel from the model limits the step after the event (E-24).
- Honors the step-rejection return flag: when an event fired inside the step, `OSDItrunc` requests `delta/8` (with a `20·CKTdelmin` termination floor) so the integrator bisects onto the discontinuity instead of extrapolating across it (E-55).

`osdi/Makefile.am`

Adds `osdiaccept.c` to the build (E-55).

---

3. Analyses — `src/spicelib/analysis/`

`dctran.c` (46 diff lines)

- Advances OSDI delay/crossing histories via the accept path and skips history corruption on rejected steps (E-1, E-6).
- Issues the dedicated `@(final_step)` evaluation at the converged end of a successful transient (E-53).

- Honors accepted-point `$finish` (ends the analysis cleanly, firing `final_step` at the requesting point), `$stop` (pauses resumably), and aborts with a clear error on `$fatal`'s `E_PANIC` instead of retrying it as nonconvergence (E-55).

`dcop.c` (9) and **`acan.c`** (10)

- Fire `@(final_step)` at their successful end (an operating point fires both step events — a single point is first *and* last) (E-53).
- An AC job's operating-point phase carries the analysis name bit (only the name — adding the reactive `CALC_*` bits would wrongly enable integration at an op), so `analysis("ac")` holds through the whole AC run per LRM 4.6.1 and `@(initial_step("ac"))` fires in AC (E-53).

`dctrcurv.c` (150) + **`include/ngspice/trcvdefs.h`** (13)

- Generic **`.dc @inst[param]`** sweeps (a new sweep code): the DC sweep previously hardcoded Vsource/Isource/Resistor/temperature, so sweeping any other device parameter was a fatal error. The generic sweep resolves `@inst[param]` through the device's own parameter tables, refreshes per point via `DEVtemperature` (for OSDI exactly the `alter` path), nests with other sweep variables, and restores the original value afterwards (E-62).
- Fires `final_step` at the last sweep point; honors a `$finish` raised mid-sweep (E-53, E-55).

`noisean.c` (37)

- A singular AC matrix during noise analysis crashed ngspice with a SIGABRT: the code ignored `NIacIter`'s return and the adjoint solve asserted on the unfactored matrix. It now aborts cleanly ("AC solution failed at ... Hz") (E-56).
- Honors deferred `$finish`/`$stop` raised at the noise operating point and fires `final_step` on success (E-55, E-53).

`span.c` (31) + **`cktspdum.c`** (14)

- 1-port S-parameter analyses enabled: the "we need at least two ports" error was over-strict (a 1-port is a plain reflection measurement; the matrix machinery is N-general) — and it was hiding the `cadjoint` crash fixed in `dense.c` (E-64).
- **`Rbase`** published into the sp plot from port 1's `z0`, making the Touchstone writers work with no manual `let Rbase = 50` incantation (E-64).
- Stock NaN fixed in **`donoise`** noise-parameter extraction: the uncorrelated noise conductance `Gu` is analytically zero for fully-correlated single-source topologies, and floating-point rounding could land `sqrt(Ycor² + Gu/Rn)`'s argument at $-10^{-18} \to$ NaN for the noise figure. Clamped to the physical range ≥ 0 — found by parity testing against an OSDI resistor that produced the exact textbook NF where the built-in returned NaN (E-63).

`cktdisto.c` (16)

- `.disto` silently reported zero distortion for OSDI devices — the distortion kernel needs higher-order Taylor coefficients the OSDI ABI cannot provide (first derivatives only), and such devices were skipped without a word. A prominent warning now names each affected OSDI device type (E-62).

4. Frontend — **src/frontend/**

plotting/pyplot.c (new) + **com_pyplot.c** (new) + **plotit.c**, **commands.c** (E-94)

A new interactive command, **pyplot**, plots simulated vectors with matplotlib — a Python counterpart to `gnuplot`. `ft_pyplot()` (`plotting/pyplot.c`) mirrors `ft_gnuplot()`: it writes a `<file>.data` table and a `<file>.py` matplotlib script and `system()`s `python3` (synchronously for a PNG, backgrounded for an interactive window). A new `"pyplot"` device arm in `plotit()` routes to it, so it accepts the same plot expressions as `plot`/`gnuplot`; the `com_pyplot()` wrapper mirrors `com_gnuplot()`, and `pyplot` is registered in both command tables. Two `set` variables tune it: `pyplot_terminal=png` renders headless (Agg) to a PNG, and `pyplot_python` picks the interpreter (default `python3`). Purely additive (E-94). The output file base name is optional (E-95): `com_pyplot()` treats the first word as a file name only when it is not a plot expression (no `(`, and not a vector by `vec_get()`), otherwise it defaults to `pyplot` — so `pyplot v(out)` plots directly (the command's minimum argument count was lowered from 2 to 1). `ft_pyplot()` also grew stacked subplots (`set pyplot_subplots=N` → `plt.subplots(nrows, 1, sharex=True)`, `N` traces per panel; `vs` stays the x-axis, so it is not a panel separator) and matplotlib style sheets (`set pyplot_style=<name>`, `dark` → `dark_background`, guarded) (E-98). The headless `pyplot_terminal` was extended from PNG-only to the vector export formats `svg` and `pdf` (`set pyplot_terminal=svg/pdf` → `fig.savefig(<file>.<fmt>)`, matplotlib picking the writer from the extension), and a figure size control was added (`set pyplot_figsize="W,H"` → `plt.subplots(..., figsize=(W, H))`; quote the value so ngspice keeps the comma) (E-99).

postcoms.c (414 diff lines) + **postcoms.h** + **commands.c** (16)

The Touchstone I/O subsystem (E-64, E-72):

- **wrsnp** (with `wrs2p` dispatching to the same handler): Touchstone v1 for any port count — the classic 2-port `S11 S21 S12 S22` column order preserved byte-identically, N $\geq$ 3 in the spec's row-major layout with at most four complex pairs per line;
- writer options `wrsnp <file> [ri|ma|db] [s|y|z] [hz|khz|mhz|ghz]`, any combination in any order: MA/DB formats, Y/Z parameters (normalized to Rbase, `Y·R` / `Z/R`, per the v1 spec), frequency units — the option line reflects every choice;
- **rdsnp**: reads any Touchstone v1 file into a new plot with a Hz `frequency` scale and complex vectors matching the `.sp` plot's conventions (MA/DB converted back, Y/Z de-normalized, the 2-port column order handled, `Rbase` published so imports round-trip) — measured data diffs against simulation in one `let` expression.

Why: E-63 showed the S-parameter *analysis* was exact but its results could not leave ngspice in the industry-standard format (`wrs2p` demanded a never-created `Rbase` vector), and nothing could bring VNA data in.

outitf.c (16)

- Integer operating-point variables recorded per point: `getSpecial` masked with `IF_REAL` only, so an integer opvar saved with `.save @n1[flag]` produced garbage in transient vectors (E-32).
- The plot-memory guard's message printed `%Id` — a Windows-only printf length modifier that is undefined behavior elsewhere and rendered the message as the numberless `"memory required (Id Bytes)"`. Now `%zu`: real byte counts on every platform (E-77).

resource.c (27)

`ft_ckspace()` — the *"approaching max data size"* warning (RSS > 95% of RSS + free RAM) — now honors **`set no_mem_check`** (the same opt-out the fatal output-size estimate in `outitf.c` respects; one variable governs both memory guards) and warns once per excursion above the threshold, re-arming below 90%, instead of repeating on every check through a long analysis (E-81).

`display.c` (1)

`#undef NODEV` before the local macro definition — the macOS SDK's `<sys/param.h>` defines an unrelated `NODEV` (E-77).

---

5. Parser and device registry — `src/spicelib/`

`parser/inpgtok.c` (10)

A **`.model`** card override silently dropped its first parameter when the type name and the opening parenthesis had no space between them (`modname devtype(p1=v1 …)`): `INPgetNetTok`'s scan did not treat `(` as a token boundary, so the type token swallowed the paren and the first parameter name. Found while verifying event-function counters whose model-card overrides mysteriously kept defaults (E-8).

`parser/inpgmod.c` (5)

`find_model_parameter()` dereferenced `*(device->numModelParms)`, which is NULL for built-ins that take no model cards (VCVS, CCCS, …) — any *referenced* `.model m vcvs()` card segfaulted, with no OSDI involved at all (an ordinary MOS instance sufficed). This was the true root of the long-documented "module named like a built-in crashes" gotcha. One NULL guard; both shapes now produce clean, located errors (E-76).

`devices/dev.c` (51)

- Duplicate device-type registration warned and skipped: `osdi_add_device()` appended every descriptor unconditionally, and the model-card lookup scans front-to-back — so a module name duplicated across two loaded libraries silently resolved to the *first* registration (loading an updated model library gave the stale device with no hint), and a module named like a built-in corrupted model creation. Registration now checks the table case-insensitively and skips duplicates loudly (E-76).
- **`pre_osdi`** path dedup: loading an already-loaded file notes it and skips — with the remedy stated: *"restart ngspice to load a recompiled file"* (E-76, E-81).

`osdi/osdiparam.c` (E-93)

Setting a fixed (**`localparam`**) OSDI parameter from the netlist used to be swallowed silently: openvaf exports every `localparam` as a parameter, its "given" flag is forced false, and the written value is overwritten by the parameter-init default — so `.model … N=8` on a frozen structural width parameter changed nothing, with no feedback. `OSDIparam`/`OSDImParam` now test the new `PARA_FLAG_FIXED` descriptor flag (set by openvaf, a free bit of the parameter `flags` field) and emit *"parameter 'N' is a fixed (localparam) value and cannot be set from the netlist; ignored"* instead of silently dropping it (E-93).

---

6. Maths — `src/maths/`

`dense/dense.c` (11 substantive lines; the file also carries a whitespace/line-ending normalization from editing)

`cadjoint()` had no 1×1 base case: its cofactor loop allocated 0×0 and then negative-sized minors, killing ngspice with `malloc: can't allocate -8 bytes` — the crash that had been hiding behind the "at least two ports" guard in `span.c`. The adjugate of `[a]` is `[1]` (the empty minor's determinant is 1 by convention), making `cinverse` of a 1×1 equal `1/a`; a 100 Ω load on a 50 Ω port now writes a proper `.s1p` with S11 = 1/3 exactly (E-64).

`sparse/spfactor.c` (6)

The Markowitz-product overflow guards compare a `double` against `LARGEST_LONG_INTEGER` (= `LONG_MAX`), which is not exactly representable as a double; the conversion is now explicit — identical semantics, intentional and warning-free (E-77).

`KLU/klu_multiply.c` (16)

`SMPmultiply`'s KLU path passed `NULL` internal↔external ordering maps to `KLU_matrix_vector_multiply`, which dereferenced them unconditionally (`&NULL[n] = 0x14` for `n = 5`) and segfaulted — so `.option linesearch` crashed under `.option klu`. The line search is the only caller of `SMPmultiply` under KLU, so this path had never been exercised. A NULL map now means the identity ordering (`ext = i + 1`, since the KLU CSR is 0-based and the RHS/Solution vectors are 1-based), making the KLU matrix-vector product — and hence the E-111 line-search residual merit — correct under KLU, with a merit sequence numerically identical to Sparse 1.3 (E-112).

`KLU/klusmp.c` (SMPcaSolve)

`SMPcaSolve` is the complex adjoint (transposed) solve used by noise and S-parameters. Its Sparse branch calls `spSolveTransposed`, but the KLU branch called the *non-transposed* `klu_z_solve` — silently wrong for any asymmetric matrix (every circuit with a transistor or controlled source), which is why `.noise` was disabled under KLU. It now calls `klu_z_tsolve` (transposed), so KLU noise matches Sparse exactly (including OSDI device models). This is the core of E-113, which also removes the KLU refusal guards in `noisean.c` / `pzan.c` (single-ended pole-zero runs under KLU; balanced-output pole-zero keeps a targeted guard).

`spicelib/analysis/cktsens.c`

Sensitivity builds an auxiliary perturbation matrix `delta_Y` (holding `∂Y/∂p`) via `SMPnewMatrix`, which allocates a plain Sparse 1.3 matrix (`delta_Y->CKTkluMODE = 0`, no `SMPkluMatrix`). It is only *multiplied* against the solution (`SMPmultiply` → `spMultiply`), never factored, and `DEVbindCSC` always binds into `ckt->CKTmatrix`, not `delta_Y` — so it is correctly Sparse in every case. But two KLU-only setup blocks were gated on the main matrix's `CKTkluMODE`, so under `.option klu` they ran and dereferenced the NULL `delta_Y->SMPkluMatrix` — a segfault on every DC/AC `.sens` deck. The blocks now gate on `delta_Y->CKTkluMODE` (its own flag, `0`), keeping `delta_Y` Sparse under KLU exactly as it already is under the default solver, while the main `Y` matrix stays KLU. DC and AC sensitivity now match Sparse exactly (E-114).

`spicelib/analysis/distoan.c`

Distortion is a complex analysis (Volterra series solved at the harmonic / intermodulation frequencies via `NIdIter` → `SMPcSolve`), but `distoan.c` had no KLU code at all: under `.option klu` the KLU matrix stayed in *real* mode, the complex solves ran against an unconverted matrix, and every harmonic came back zero — a silent wrong answer. The fix mirrors `acan.c`: before the solve loop it converts the matrix to complex (`DEVbindCSCComplex` per device + `KLUmatrixIsComplex`), and on exit converts back to real (`DEVbindCSCComplexToReal`) so a later real analysis in the same session is unaffected. KLU distortion now matches Sparse bit-for-bit (single-tone, two-tone intermodulation, and OSDI models) (E-115).

---

7. Build system

`xspice/icm/makedefs.in` (4)

The XSPICE codemodel (`.cm`) link rule hardcoded the pre-macOS-10.3 idiom `-bundle -flat_namespace -undefined suppress`, deprecated loudly by the modern linker on all 14 codemodel links (CI macOS builds included). Now `-bundle -undefined dynamic_lookup` — the modern spelling for plugins whose host symbols resolve at load time (E-77).

Autotools regeneration (the ~100 **`Makefile.in`** files + **`config.h.in`**)

Regenerated by `./autogen.sh` with libtool 2.5.4 during the zero-warning work — a build tree whose `configure` predates libtool 2.4.7 selects the deprecated darwin `-undefined suppress` flag whenever `MACOSX_DEPLOYMENT_TARGET` is unset. No hand-written content (E-77).

---

8. Per-enhancement index

Every enhancement that touched ngspice, oldest first:

Enhancement	ngspice files	One line
E-1	osdi.h, osdidefs.h, osdiitf.h, osdiregistry.c, osdiload.c, osdisetup.c, dctran.c (+accept path)	`absdelay()`: synthetic delay rows + waveform history
E-6	osdi.h, osdidefs.h, osdiitf.h, osdiregistry.c, osdiload.c	`last_crossing()`: history + interpolated crossings
E-8	parser/inpgtok.c	`.model` card first-parameter drop fix
E-24	osditrunc.c	`$discontinuity` next-step clamp
E-25	osdi callbacks (get_simparams)	expose `analysis_name` + `simulator` simparams
E-32	outitf.c	integer opvars recorded per point
E-42	osdinoise.c	coherent same-named noise grouping (LRM 4.6.4)
E-45	osdi.h (ABI 0.5), osdisetup.c	Verilog-A nodesets applied to the solver
E-51	osdi.h (ABI 0.6), osdiacld.c	full `ac_stim` AC-RHS injection
E-53	dctran.c, dcop.c, dctrcurv.c, acan.c, noisean.c, osdiload.c	`@(final_step)` + analysis-name phase bits
E-54	osdi.h (ABI 0.7), osdinoise.c, osdiload.c, osdiacld.c, osdiregistry.c	node-free complex noise factors, stride-2 pairs
E-55	osdiaccept.c (new), osdiload.c, osditrunc.c, osdiinit.c, dctran.c, dctrcurv.c, osdiitf.h, Makefile.am	`$finish`/`$stop`/`$fatal` honored; `$discontinuity` step rejection
E-56	noisean.c, osdisetup.c	noise SIGABRT fix; setup-rejection diagnostics
E-62	dctrcurv.c, trcvdefs.h, cktdisto.c	generic `.dc @inst[param]` sweeps; `.disto` warning
E-63	span.c	`donoise` NaN clamp (stock defect, found by parity)
E-64	span.c, cktspdum.c, dense.c, postcoms.c/.h, commands.c	Touchstone export, auto-Rbase, 1-port `.sp`, `cadjoint` 1×1
E-72	postcoms.c/.h, commands.c	Touchstone round 2: MA/DB/Y/Z/units + `rdsnp` reader

Enhancement	ngspice files	One line
E-76	dev.c, inpgmod.c	duplicate-registration warning, load dedup, stock `.model` segfault fix
E-77	osdidefs.h, display.c, outitf.c, spfactor.c, makedefs.in (+autotools regen)	zero-warning build, 33 → 0
E-80	osdiinit.c	`dtemp` instance-parameter alias
E-81	resource.c, dev.c	memory-warning opt-out + once-per-excursion; reload-note hint
E-93	osdi.h, osdiparam.c	warn (not silently ignore) when a netlist sets a `PARA_FLAG_FIXED` (`localparam`) OSDI parameter
E-94	plotting/pyplot.c (new), com_pyplot.c (new), plotit.c, commands.c	new `pyplot` command — plot vectors with matplotlib (a Python `gnuplot`)
E-95	com_pyplot.c, commands.c	make the `pyplot` output file name optional (defaults to `pyplot`)
E-98	plotting/pyplot.c	`pyplot` stacked subplots (`pyplot_subplots=N`) + matplotlib style sheets (`pyplot_style`)
E-99	plotting/pyplot.c	`pyplot` vector export formats (`pyplot_terminal=svg/pdf`) + figure size (`pyplot_figsize`)
E-110	optdefs.h, tskdefs.h, cktsopt.c, cktntask.c	`.option errpreset=conservative|moderate|liberal` — one knob for a coordinated tolerance/robustness set; explicit options override regardless of order (`moderate` = historical defaults)
E-111	optdefs.h, tskdefs.h, cktdefs.h, cktsopt.c, cktdojob.c, cktntask.c, cktdest.c, niiter.c	`.option linesearch` — globalized (damped) Newton via Armijo backtracking on a new KCL-residual merit `‖F‖ =‖G·x−b‖`; result-neutral, off by default
E-112	maths/KLU/klu_multiply.c	KLU support for `.option linesearch` — `SMPmultiply`'s KLU path passed NULL ordering maps that `klu_matrix_vector_multiply` dereferenced (SIGSEGV); NULL now means identity ordering. Line search verified merit-identical KLU vs Sparse
E-113	maths/KLU/klusmp.c, spicelib/analysis/noisean.c, pzan.c	KLU support for noise + single-ended pole-zero — `SMPcaSolve`'s adjoint KLU branch used the non-transposed solve (silently wrong noise on asymmetric matrices); now `klu_z_tsolve`. Guards removed; balanced-output pz keeps a targeted guard

Enhancement	ngspice files	One line
E-114	spicelib/analysis/cktsens.c	KLU support for sensitivity — the auxiliary perturbation matrix `delta_Y` is Sparse, but two KLU setup blocks gated on the *main* matrix's flag dereferenced its NULL `SMPkluMatrix` (segfault on every DC/AC `.sens`); now gated on `delta_Y`'s own flag. DC/AC `.sens` match Sparse exactly
E-115	spicelib/analysis/distoan.c	KLU support for distortion — `.disto` is a complex analysis but `distoan.c` had no KLU code, so under KLU the matrix stayed real and every harmonic came back zero (silent wrong answer); now converts real↔complex around the solve loop like `acan.c`. Matches Sparse bit-for-bit
E-116	osdi/osdisetup.c	KLU wrong-DC fix — an OSDI internal node with no Jacobian entry (a decoupled `ground` reference) was given an all-zero solver row that made the KLU matrix structurally singular; now tied to ground. Fixes the `groundcontrib` and `hierbranch` KLU_XFAILs
E-117	configure.ac (+`configure`), spicelib/analysis/dcpss.c	Periodic steady state (PSS) productionized — built by default (was experimental `--enable-pss`, so `.pss` was unimplemented in shipped builds); ~230 lines of shooting-loop trace routed through `set ngdebug` (232 → 31 lines by default); fail-fast KLU guard (PSS hangs under KLU) directing `.pss` to `.option sparse`
E-118	spicelib/analysis/dcpss.c	PSS runs under KLU — the shooting loop hung under KLU (`klu_refactor`'s reused pivots inflated the truncation error into a ~20M-step timestep explosion); forcing a full re-factor (`NISHOULDREORDER`) each PSS step makes KLU converge to the same result as Sparse. E-117's Sparse-only guard removed
E-119	pssdefs.h, spicelib/analysis/dcpss.c	Retain the PSS periodic operating point — PSS sampled the node voltages per period for its DFT then freed them, and never captured device states; now the voltages and `CKTstate0` states are captured per sample and retained on the `PSSan` job (with frequency + dims) as the substrate for the periodic small-signal suite (PAC/pnoise/PXF). Self-checks that the retained samples reproduce the fundamental

<table>
<tr><th>Enhancement</th><th>ngspice files</th><th>One line</th></tr>
<tr><td>E-120</td><td>spicelib/analysis/dcpss.c</td><td>Periodic small-signal Jacobian harmonics — walk the retained operating point, re-linearize each sample (CKTload MODEINITSMSIG) and stamp G+jC (CKTacLoad ω=1), read the osc-node diagonal → g(t), c(t), DFT to harmonics. The blocks the PAC conversion matrix is built from. (Fixed a complex-mode-not-set bug where C(t) accumulated across samples.) Verified: RC gives G=1/R1, C=C1, no harmonics</td></tr>
<tr><td>E-121</td><td>spicelib/analysis/dcpss.c</td><td>PAC conversion-matrix engine — extend E-120 from the osc diagonal to every matrix nonzero, complex-DFT to harmonics G_k, C_k, assemble the (2M+1)N harmonic conversion matrix H_{nm}=G_{n-m}+jω_m·C_{n-m}, inject a unit current at the osc node in sideband 0 and solve by dense complex LU (pss_csolve), report the sideband conversion gains. The numerical heart of PAC/pnoise/PXF. Verified: linear RC gives sideband-0 = AC driving-point ‘</td></tr>
<tr><td>E-122</td><td>include/ngspice/pssdefs.h, spicelib/analysis/psssetp.c, spicelib/parser/inp2dot.c, spicelib/analysis/dcpss.c</td><td>.pac command (periodic AC) — the user-facing analysis on the E-121 engine. .pac Fguess StabTime OscNode Points Harmonics SC_iter Steady_coeff <dec|oct|lin> Npts Fstart Fstop runs PSS then sweeps the input frequency, solving the conversion matrix at each point and emitting the 0-th-sideband node responses as a complex PAC Analysis plot (print/plot/wrdata). Reuses the PSS analysis via a PSSdoPAC flag; engine factored into pac_extract_harmonics (once) + pac_solve_at (per freq). Verified: swept sideband-0 |b(f)| == analytic AC driving-point |Z(f)| to 1.6e−7 across 10 kHz–1 MHz</td></tr>
</table>

Enhancement	ngspice files	One line
E-123	include/ngspice/pssdefs.h, spicelib/analysis/psssetp.c, spicelib/parser/inp2dot.c, spicelib/analysis/dcpss.c	finish `.pac` — (1) source-referenced stimulus: capture the netlist AC-source RHS from `CKTacLoad` as the sideband-0 stimulus `B_0` (a periodic-AC transfer/conversion gain), unit-current-at-osc-node fallback; (2) multi-sideband output: optional trailing `maxsideband Ksb` emits every conversion sideband `f_in+k·f0` (`k=−Ksb..Ksb`) as its own vector — sideband 0 keeps the node name, others `<node>_usb<k>`/`<node>_lsb<k>` via `IFnewUid`. Verified: RC with `V1 AC 1` gives sideband-0 = low-pass transfer `1/√(1+(2πfRC)^2)` (0.998→0.157) with `b_usb1`/`b_lsb1` ~0 (no conversion)
E-124	include/ngspice/pssdefs.h, spicelib/analysis/psssetp.c, spicelib/parser/inp2dot.c, spicelib/analysis/dcpss.c	`.pnoise` command (periodic noise) — fold each device's noise through the conversion matrix. `pac_solve_adjoint` solves `Hᵀ Ψ = e_{out,0}`; `pnoise_sweep` loads the sideband-`k` adjoint into `CKTrhs`/`CKTirhs` and calls the existing device noise routines (`NevalSrc`, OSDI `load_noise`) once per sideband, accumulating `Σ_k S·|ΔΨ_k|^2` — reuses every device noise model via a minimal local `NOISEAN` context. `.pnoise <pss> OutNode InSrc <dec|oct|lin> Np Fstart Fstop`; emits `onoise_spectrum`/`inoise_spectrum`. Verified: linear RC reduces exactly to `.noise` (`4kTR/(1+(2πfRC)^2)`), matches the `.noise` reference to every digit)
E-125	include/ngspice/pssdefs.h, spicelib/analysis/psssetp.c, spicelib/parser/inp2dot.c, spicelib/analysis/dcpss.c	`.pxf` command (periodic transfer function) — the adjoint of PAC, completing the PSS→PAC→Pnoise→PXF suite. `pxf_sweep` solves `Hᵀ Ψ = e_{out,0}` per frequency and dots each sideband block with the AC-source pattern `B0` → `xf_k=Σ_j Ψ_k(j)·B0(j)`, the input→output transfer at each sideband. `.pxf <pss> OutNode <dec|oct|lin> Np Fstart Fstop [maxsb]`; emits `xf` + `xf_usb<k>`/`xf_lsb<k>`. Verified: by the identity `(H⁻¹B)_out=(H⁻ᵀe_out)ᵀB` the sideband-0 transfer is bit-identical to the PAC response (0.998→0.157 low-pass), conversion sidebands ~2e−16

Enhancement	ngspice files	One line
E-126	include/ngspice/pssdefs.h, spicelib/analysis/psssetp.c, spicelib/parser/inp2dot.c, spicelib/analysis/dcpss.c	cyclostationary noise — `.pnoise ... cyclo` evaluates each device's noise PSD `S(t)` at every PSS sample's bias (`CKTload` per sample) and folds it through the time-domain adjoint transfer `A_s(j)=Σ_k Ψ_k(j)e^{j2πks/P}`, averaging: `onoise(f)=(1/P)Σ_s S(t_s)·|ΔA_s|²`. Reuses the device noise routines. Verified: (1) reduces exactly to `.noise` on the linear RC (bias-independent thermal → Parseval); (2) a flicker resistor carrying the RC current gives `onoise·f = R1²·KF·⟨I²⟩ = 4.88e-10` (uses the period-average `⟨I²⟩`, matched to 5 digits)
E-127	include/ngspice/{optdefs,tskdefs,cktdefs}.h, spicelib/analysis/{cktsopt,cktntask,cktdojob,cktop,cktdest}.c, maths/ni/niiter.c	pseudo-transient continuation — `.option ptcont` embeds `f(x)=0` in a backward-Euler pseudo-transient `f(x)+Gps·(x-x_prev)=0` and marches `dτ` small→large (`Gps`→`0`). `Gps` diagonal via the gmin path; the `Gps·x_prev` RHS coupling (added in `NIiter`) makes each step a stable-trajectory move, not a static gmin step. New `pseudo_transient` in the `CKTop` cascade, off by default. Verified under KLU+Sparse: result-neutral on normal circuits; on a stiff no-limiting exp, reaches the correct DC `0.837922 V` where plain Newton lands on a spurious `70.5 V` (21/21)
E-128	include/ngspice/{optdefs,tskdefs,cktdefs}.h, spicelib/analysis/{cktsopt,cktntask,cktdojob}.c, spicelib/analysis/dctran.c	LTE-based dynamic integration-order control — `.option dynorder` selects the Gear order per step from the local-truncation-error limit instead of the stock 1↔2 toggle, so orders 3–6 (already coded in `NIcomCof` but never used) are actually exercised. The selector compares the raw LTE-limited step (uncapped `CKTtrunc`) at the current order and its ±1 neighbours and moves with hysteresis (1.2×), a settling hold after each change (let the BDF divided differences rebuild), and an order-dependent step-growth cap (2× at order ≤3 → 1.3× at 6). Off by default; bounded by `maxord`; inert at the default `maxord=2`. Stiff-transient guards (post-breakpoint order hold + leaky-bucket rejection-rate order drop) keep it from collapsing the step on a violent slew. Verified under KLU+Sparse: RC decay 3–5× fewer steps at matched accuracy with monotone error; smooth RLC ringdown 8.9× fewer steps AND more accurate (0.13 % vs 0.34 %); nonlinear rectifier matches stock to 5 sig figs; the transistor-level µA741 ±5 V slew completes under dynorder and matches fixed Gear-2

Enhancement	ngspice files	One line
E-129	frontend/outitf.c	sweep progress bar — the throttled Reference value status line (redrawn in place every 0.25 s during a sweep) now carries a live progress bar + percentage, e.g. Reference value : 5.91926e-04 [=================] 74%. outp_progress_frac() computes the 0–1 fraction per analysis: transient from (CKTtime-TSTART)/(TSTOP-TSTART), AC/noise from the frequency's linear/log position in the band, DC from the accepted-point count over the nested step-count product; analyses with no span (op, …) keep the plain line. Fixed-width (no stale chars), stdout status-line only (never in the rawfile/wrdata), same !ft_norefprint && !cp_background gating. Verified 22/22: printed % matches the analytic sweep fraction to 0.5 % across tran/AC/DC/noise; op shows no bar
E-130	frontend/com_optimize.{c,h}, commands.c, options.c, include/ngspice/fteext.h, spicelib/analysis/cktdojob.c, frontend/outitf.c	built-in Nelder-Mead optimizer — optimize -param <name> <init> <lo> <hi> [-param ...] -analysis <cmd> -minimize <expr> [-maxiter N] [-tol T] [-verbose] varies device/alter parameters, re-runs an analysis, and minimizes a scalar objective expression via a derivative-free downhill simplex in normalized [0,1] space (scale-invariant across orders-of-magnitude params). Sub-commands dispatched synchronously through cp_coms (not the deferring re-entrant cp_evloop); the hundreds of inner analyses are silenced via a new ft_optimizing flag gating the banner/row-count/E-129 progress bar at source (-verbose to show). Verified 9/9 against analytic optima: DC divider R1→2333.3 Ω, AC low-pass R1→2756.6 Ω, 2-D compound objective R1=3k/R2=2k exactly

Enhancement	ngspice files	One line
E-131	frontend/com_checkpoint.{c,h}, commands.c, com_commands.h, Makefile.am, spicelib/analysis/dctran.c, include/ngspice/cktdefs.h	transient checkpoint / restart — new `savestate <file>` / `loadstate <file>` commands serialize the full transient integration state (solution vector `CKTrhsOld`, device state history `CKTstates[]`, time/step/order/mode, pending breakpoints) to a binary file and resume it, including in a fresh process — so a long run survives a crash, splits across sessions, or moves between machines (stock ngspice could only `resume` in memory). `DCtran()` gains a `CKTcheckpoint`-gated branch that opens a fresh output plot (no live plot to `666`-relink across a reload), initializes the XSPICE breakpoint markers, and fixes up the breakpoint list before jumping into the stepping loop; the rhs length is keyed off `SMPmatSize+1` (not `CKTmaxEqNum`). A stored signature rejects a mismatched circuit; Sparse-solver only (KLU rejected with a clear message). Verified 19/19: resumed waveform bit-identical to an uninterrupted run for RC-step/pulse/diode built-ins and ~2e-7 for a compiled OSDI diode, across a separate process

Enhancement	ngspice files	One line
E-132	spicelib/analysis/dcpss.c, spicelib/parser/inp2dot.c, spicelib/analysis/psssetp.c, include/ngspice/pssdefs.h	periodic S-parameters (`.psp`) — small-signal scattering parameters around a PSS operating point, including conversion between the input frequency and its sidebands `f_in+k·f0` (mixers / switched circuits, where a static-DC `.sp` cannot see the conversion). Sits on the PSS→conversion-matrix suite (E-117–126): after PSS, `psp_sweep` excites each RF port (`portnum/z0`, the `.sp` framework) by driving its branch source (V=1, like `.sp`'s `VSRCspupdate`) through the shared `(2M+1)N` conversion matrix, reads per-sideband port waves in the same Kurosawa power-wave convention, and forms `S^(k)=B^(k)·A^−1` (dense-complex `cinverse/cmultiply`). `pac_solve_at`'s matrix assembly factored into a reusable `pac_build_matrix`; `PSSdoPSP` flag + `psp` PSS param + `dot_psp` card. Because `S=B·A^−1` is excitation-basis-invariant, sideband 0 reduces exactly to `.sp` for a time-invariant network. Runs under both linear solvers (the conversion matrix is a standalone dense LU; PSS runs under both since E-118). Verified 8/8: sideband-0 matches `.sp` to ~1e-16 for 1/2/3-port resistive + reactive networks (magnitude and phase) incl. OSDI Verilog-A devices (G + reactive `ddt` stamps), conversion sidebands correctly ~0

Enhancement	ngspice files	One line
E-133	frontend/com_qpss.{c,h}, commands.c, com_commands.h, Makefile.am	quasi-periodic (two-tone) steady state (qpss) — qpss <expr> <f1> <f2> [periods] [maxorder] computes the two-tone steady-state spectrum: every mixing product k1·f1+k2·f2 including third-order intermodulation (IM3 at 2f1-f2 / 2f2-f1) that single-tone AC/PSS cannot show. For commensurate tones (common beat fb=gcd(f1,f2)) it runs an ordinary transient over a few beat periods to reach steady state, then evaluates the Fourier coefficient directly at each exact intermod frequency (a direct DFT, exact — no FFT-bin rounding) and labels it by the 2-D index (k1,k2). Deliberately not a slow beat-frequency shooting PSS; a front-end command driving a transient, so solver-independent and works with built-in and OSDI devices. Verified 11/11: analytic fundamentals/IM3/3f of a cubic, no even-order products, the 3:1 IP3 slope law (fund ×2, IM3 ×8 per 2× drive), beat-frequency derivation, and an OSDI cubic matching the built-in

Enhancement	ngspice files	One line
E-134	spicelib/analysis/dcpss.c, frontend/com_hb.{c,h}, commands.c, com_commands.h, Makefile.am, include/ngspice/cktdefs.h	Harmonic Balance (`hb <f0> <K>`) — solves the periodic steady state in the frequency domain by Newton (each node a truncated Fourier series), instead of integrating in time; the real analysis behind ngspice's unimplemented `WITH_HB` stub. Residual `F_k = I_R,k(V) + [dq/dt]_k − Is_k = 0` with the E-121 `(2K+1)N` conversion matrix as the exact Jacobian. Per iteration: inverse-DFT V→v(t_s); drive DC+AC device loads at those voltages for the resistive current `I_R = G*v − b` (companion `b` saved before acLoad, so it's the ACTUAL current not the tangent `G*v`), and `G(t)/C(t)`; DFT; dense complex Newton (`pss_csolve`). Nonlinear reactive with NO charge extraction: `dq/dt = C(v)*v'`, so the reactive current is the conversion matrix's `jwC` term applied to V. Solver-independent (KLU + Sparse): the dense complex Newton is HB's own; the sparse solver only *reads* `G(t)/C(t)` off the device matrix, so `hb_extract` carries the same `#ifdef KLU` complex-CSC binding (`DEVbindCSCComplex`) as the PAC extraction, and `com_hb` copies the task's KLU mode before building so a bare `hb` honours `.option klu` — verified bit-identical under both. Built-in + OSDI. Verified 8/8 vs transient/`fourier`, quadratic convergence: nonlinear R (analytic 3rd harmonic), R+C, a real diode rectifier (junction limiting), OSDI varactor `Q(v)` 2nd harmonic, KLU==Sparse parity

Enhancement	ngspice files	One line
E-135	spicelib/analysis/dcpss.c, examples/hb_examples/verify_hb.py	HB source-stepping continuation — makes E-134 Harmonic Balance robust on strongly-driven circuits where a cold full-strength Newton diverges (`|F|`→`1e69`). `HBanalyze`'s Newton loop is wrapped in an adaptive homotopy: every independent source scaled by `λ: 0`→`1`, each level warm-started from the last converged `V`. The residual becomes `F = I_R + I_C − λ·Is`; on level success `V` is checkpointed and `dλ` grows (×1.7), on failure (Newton exhausted, residual non-finite, or singular Jacobian) `V` is restored and `dλ` halves; `dλ<1e−5` → reported as no reachable steady state. First level is full strength (`dλ=1`) so easy circuits converge at `λ=1` immediately, bit-identical to the plain solve. Automatic (no new syntax); `set hb_verbose` shows the `λ` ramp, summary reports iterations + continuation steps. Verified 9/9: a strongly-driven diode rectifier (5 V into 20 Ω, sharp `IS=1e−14`) diverges cold but converges in 3 continuation steps, matching transient `fourier` to <0.1% for DC/f0/2f0; the 8 existing HB checks stay bit-identical

Enhancement	ngspice files	One line
E-136	spicelib/analysis/dcpss.c, frontend/com_qpss.c, frontend/commands.c, include/ngspice/cktdefs.h, examples/qpss_examples/verify_qpss_hb.py	two-tone Harmonic Balance (`qpss <expr> <f1> <f2> hb [K1] [K2]`) — the TRUE, incommensurate-capable quasi-periodic steady state, a frequency-domain HB engine alongside the E-133 transient `qpss` (which is unchanged, commensurate-only). Each node is a 2-D Fourier series `v(t)=ΣΣ V_{k1,k2} e^{j(k1ω1+k2ω2)t}`; `QPSShb` samples devices on a 2-D phase grid `(θ1,θ2)` — time never appears, so incommensurate tones (no beat period) just work — and 2-D DFTs to the conversion matrix `H_{(n),(m)}=G_{n-m}+jω_m·C_{n-m}` (`qp_build_matrix`), Newton-solved by `pss_csolve` with the E-135 source stepping. Sources captured by an oversampled least-squares APFT `(Γ^H Γ) Is = Γ^H b` (a square Vandermonde is catastrophically ill-conditioned past a few harmonics). Reactive needs NO charge extraction (`dq/dt=C(v)v'`); the converged operating point is retained (`qpss_hb_saved`) for `qpac` (E-137). Solver-independent (KLU + Sparse) — same `#ifdef KLU` complex-CSC binding as PAC/HB. Built-in + OSDI. Verified 7/7: analytic cubic `|IM3(2,-1)|/|3rd(3,0)|=3`, even products ~0, 3:1 IP3 slope, incommensurate □2 tones (E-133 cannot), HB==transient (commensurate), KLU==Sparse; E-133 `verify_qpss.py` stays 11/11

Enhancement	ngspice files	One line
E-137	spicelib/analysis/dcpss.c, frontend/com_qpac.{c,h}, commands.c, com_commands.h, Makefile.am, include/ngspice/cktdefs.h, examples/qpss_examples/verify_qpac.py	two-tone small-signal QPAC (`qpac <f_in>`) — completes the quasi-periodic gap: the two-tone analogue of PAC. Run after `qpss … hb`, `QPACanalyze` injects a small signal at `f_in` around the retained QPSS operating point (`qpss_hb_saved`) and reports the response at every sideband `f_in+k1·f1+k2·f2`, mixing it through the SAME 2-D conversion matrix `H_{(n),(m)}=G_{n-m}+jω_m·C_{n-m}` (`qp_build_matrix` at `f_in`) that the QPSS Newton used as its Jacobian. Stimulus placed in the `(0,0)` sideband — a netlist AC-source RHS `B0` (captured bias-independent at the op-point) or a unit-current fallback — one dense `pss_csolve`. Adds no device evaluation → solver-independent (KLU + Sparse) by construction. Verified 7/7: reduce-to-AC (pump→0 ⍰ direct `(0,0)` = plain `.ac` response = R, sidebands vanish), v^2-pump conversion ratio `|(1,1)|/|(2,0)|=2`, equal-tone symmetry, clean no-op-point error, KLU==Sparse; E-136 (7/7) + E-133 (11/11) unaffected

Enhancement	ngspice files	One line
E-138	spicelib/analysis/dcpss.c, frontend/com_qpnoise.{c,h}, commands.c, com_commands.h, Makefile.am, include/ngspice/cktdefs.h, examples/qpss_examples/verify_qpnoise.py	two-tone small-signal QPnoise (`qpnoise <output_node> <f_in>`) — quasi-periodic noise, the two-tone analogue of pnoise (E-124), on the retained `qpss … hb` operating point. Reports output + input-referred noise density at `f_in`, folding every device's noise over all sidebands `f_in+k1·f1+k2·f2` (mixer/PA noise conversion invisible to a static `.noise`). `QPnoiseAnalyze` biases the devices at the QPSS operating point, then `qp_solve_adjoint` transposes the 2-D conversion matrix (`qp_build_matrix`) and solves $H^T \Psi$ = `e_{out,(0,0)}` — one adjoint gives the transimpedance from every (node, sideband) to the output; each device's `DEVnoise` computes $S\cdot\lvert\Psi\rvert^2$ (reading the transfer from `CKTrhs`/`CKTirhs`) and the sum over all Nh harmonics is the folded onoise; input-referred via the QPAC gain. Reads only retained data → solver-independent (KLU + Sparse) by construction. With no pump the conversion matrix is block-diagonal so only `(0,0)` survives □ reduces to `.noise`. Verified 6/6: reduce-to-noise (pump→0 □ onoise == plain `.noise` == `4kTR` exactly), conversion active under pump, `inoise=onoise/gain`^{2}, clean no-op-point error, KLU==Sparse; E-133 (11/11) + E-136 (7/7) + E-137 (7/7) unaffected

Enhancement	ngspice files	One line
E-139	spicelib/analysis/dcpss.c, frontend/com_qpnoise.c, frontend/commands.c, include/ngspice/cktdefs.h, examples/qpss_examples/verify_qpnoise.py	cyclostationary QPnoise (qpnoise <output_node> <f_in> cyclo) — upgrades E-138 to a time-varying device PSD. Under a two-tone pump a device's bias swings over the period (a diode's shot noise $2qI_D(t)$ spikes when it conducts), so QPnoiseAnalyze gains a cyclo branch using the identity $\text{onoise} = (1/P)\cdot\Sigma_s S(t_s)\cdot\lvert A_s\rvert^2$, where $A_s(j) = \Sigma_{(k1,k2)} \Psi_{(k1,k2)}(j)\cdot e^{j2\pi(k1\ s1/P1+k2\ s2/P2)}$ is the inverse 2-D DFT of the adjoint transfers (the time-domain transimpedance at phase sample s). Each sample re-biases the devices at the retained op-point (qp_synth, P1/P2 now stored in qp_harm) with per-sample junction settling (the E-134 MODEINITFLOAT walk — else a limited diode reports a stale bias and the "cyclostationary" result collapses onto the stationary one), evaluates $S(t_s)$, folds through A_s, and averages. By Parseval reduces to the stationary sum (and .noise) when S is constant. Verified 10/10 (6 E-138 + 4 cyclo): cyclo reduce-to-noise, Parseval (bias-independent thermal PSD [illegible] cyclo==stationary even under pump), a hard-pumped diode where cyclo differs from stationary by ~8× (switching-mixer cyclostationary enhancement, visible only with the settling), cyclo KLU==Sparse; E-133/136/137 unaffected

Enhancement	ngspice files	One line
E-140	spicelib/analysis/dcpss.c, frontend/com_hbosc.{c,h}, commands.c, com_commands.h, Makefile.am, include/ngspice/cktdefs.h, examples/phasenoise_examples/verify_phasenoise.py	oscillator phase noise (`hbosc <oscnode> <K> [fguess] [tstab]` + `phasenoise <fstart> <fstop> [pts]`) — closes the periodic/phase-noise gap with the phase-noise piece. `HBOSCanalyze` is an autonomous harmonic balance: an oscillator has no source, so `F(V)=I_R+[dq/dt]=0` is solved for the harmonics AND the unknown oscillation frequency `ω0`. The conversion matrix `dF/dV=H` is singular (right null space = phase mode `u_k=jk V_k`, since the oscillator's phase is free), so Newton runs on the bordered system `[H, ∂F/∂ω0; u*ᵀ, 0]` (nonsingular by bordering, `∂F/∂ω0=I_C/ω0`, gauge row `u*`), transient-seeded (`hbosc` runs a short transient from the deck's `.ic`, reads amplitude + zero-crossing frequency), converging quadratically (LC osc: 4 iters to `|F|=3e−12`; inductors fit the `G+jωC` matrix via branch-current MNA). `PhaseNoiseAnalyze` builds the adjoint of `H` at OFFSET `df` with the unit at the carrier sideband (m=1) and folds device noise (`DEVnoise`); as `df`→`0` the limit-cycle matrix goes singular through the phase mode so the transimpedance diverges as `1/df` → folded noise `1/df²`, normalized to carrier power `2|V1|²` → `L(df)` in dBc/Hz. Verified 8/8 on an LC oscillator: autonomous HB converges to `f0`+describing-function amplitude, `L(df)` has the −20 dB/dec skirt near carrier flattening into the noise floor at a physical level (≈−147 dBc/Hz @1kHz), thermal scaling **L∝T** (2×T → +3 dB exactly, pinning the absolute), clean no-op-point error, KLU==Sparse; pnoise (9/9) + qpnoise (10/10) unaffected

Enhancement	ngspice files	One line
E-141	spicelib/analysis/dcpss.c, frontend/com_qpxf.{c,h}, commands.c, com_commands.h, Makefile.am, include/ngspice/cktdefs.h, examples/qpss_examples/verify_qpxf.py	two-tone small-signal QPXF (qpxf <output_node> <f_in>) — the quasi-periodic transfer function, the ADJOINT of QPAC (E-137), completing the two-tone small-signal suite (QPSS→QPAC→QPnoise→QPXF ≡ PSS→PAC→pnoise→PXF). QPXFanalyze runs one adjoint solve H^T Ψ = e_{out,(0,0)} (qp_solve_adjoint, reused from QPnoise E-138) and dots each sideband block of Ψ with the netlist AC-source pattern B0 (the QPAC stimulus, captured at the op-point) → the transfer H_{(k1,k2)} = Σ_j Ψ_{(k1,k2),j}·B0_j from an input at every sideband f_in+k1·f1+k2·f2 to the output, all from ONE adjoint. By the reciprocity identity $(H^{-1}B)$_out=$(H^{-T}$e_out$)^T$B the sideband-(0,0) transfer is bit-identical to the QPAC response (the PXF↔PAC cross-check, E-125). Reads only retained data → solver-independent (KLU + Sparse). Verified 6/6: reciprocity (QPXF(0,0)==QPAC(0,0) bit-identical), conversion-sideband reciprocity, reduce-to-XF (pump→0 □ (0,0)=plain transfer=R, sidebands vanish), clean no-op-point error, KLU==Sparse; QPSS 11/11 + QPSS-HB 7/7 + QPAC 7/7 + QPnoise 10/10 unaffected

<table>
<tr><th>Enhancement</th><th>ngspice files</th><th>One line</th></tr>
<tr><td>E-142</td><td>spicelib/analysis/dcpss.c, frontend/com_qpac.{c,h}, com_qpnoise.c, com_qpxf.c, commands.c, include/ngspice/cktdefs.h, examples/qpss_examples/verify_qpss_sweep.py</td><td>input-frequency sweep for the two-tone small-signal analyses. `qpac/qpnoise/qpxf` gain a `<dec|oct|lin> N fstart fstop` sweep of `f_in` that emits a plottable ngspice plot (conversion gain / noise figure / image-rejection curves vs frequency), matching how `.ac/.pnoise/.pxf` sweep. `QPACsweep/QPnoiseSweep/QPXFsweep` step `f_in` and reuse the exact single-frequency solve per point: `qpac` records per-node `|(0,0) response|`, `qpnoise` records `onoise_spectrum/inoise_spectrum`, `qpxf` records `xf` (in-band `(0,0)`) + `xf_conv` (total conversion `√Σ|H_{sb≠0}|²`). The analysis fills plain arrays; the front-end builds the plot with a `frequency` scale via the nutmeg vector API (`plot_alloc/dvec_alloc/vec_new`) — the correct layer for a front-end command (the analysis OUTp framework dereferences a live analysis job's `JOBtype`, which a front-end command lacks → was crashing). Shared helpers `qp_steptype/qp_sweep_maxpts/qp_emit_plot`. Verified 5/5: swept value at 0.3G == single-frequency `qpac/qpxf/qpnoise` (machine precision), reactive roll-off vs `f_in`, correct point count; single-frequency QPAC 7/7 + QPnoise 10/10 + QPXF 6/6 unaffected</td></tr>
</table>

Enhancement	ngspice files	One line
E-143	frontend/com_optimize.c, frontend/commands.c, examples/optimize_examples/verify_optimize.py, examples/optimize_examples/optimize_lsq_demo.cir	gradient least-squares curve fitting for **`optimize`**. The built-in optimizer (E-130, derivative-free Nelder-Mead on a scalar `-minimize`) gains a least-squares mode: one or more weighted `-target <expr> <value> [<weight>]` measurements, optionally spread over several `-analysis` stages (each `-analysis` opens a stage; following `-targets` attach to it, so a single fit can combine e.g. a DC operating point AND an AC response), are fitted with Levenberg-Marquardt — a forward/backward finite-difference Jacobian `J`, normal equations $A=J^\top J$, $g=J^\top r$, damped solve $(A+\lambda\cdot\mathrm{diag}(A))\delta=-g$ via a small partial-pivot Gauss solver, standard λ up/down loop. Exploiting the sum-of-squares structure it converges in far fewer analysis runs than the simplex (27 vs 67 evals on a two-target RC fit). `-method nm|lm` overrides the auto choice (LS$\to$lm, scalar$\to$nm); the scalar `-minimize`/Nelder-Mead path is unchanged; `-minimize` and `-target` are mutually exclusive. Parameters are still applied in place with `alter` (no re-source) and the search runs in normalized [0,1] space. All changes in `com_optimize.c` (+ one help string); no new files, no ABI change. Verified 23/23 (E-130 checks [1]–[5] + E-143 [6]–[10]): 1-param/3-target/3-stage RC magnitude fit, 2-param DC+AC multi-analysis fit (R1=3k,R2=2k), LM-fewer-evals-than-NM, OSDI/Verilog-A diode extraction (recover both `is`$\to$1e-14 and `n`$\to$1.2 from two measured I-V points by weighted least squares), and input validation (lm-without-target, minimize+target, multi-analysis+scalar all rejected)

Enhancement	ngspice files	One line
E-144	frontend/com_optimize.c, frontend/inp.c, frontend/commands.c, examples/optimize_examples/verify_optimize.py, examples/optimize_examples/optimize_dparam_demo.cir	**`optimize`** tunes symbolic **`.param`** values via a new knob kind `-dparam`. `-param` remains an `alter` target (device/instance, changed in place); `-dparam` names a netlist `.param` symbol (e.g. `.param w=1u`, `R1={500*k}`) which — being expanded at parse time — is changed with `alterparam <name>=<value>` + a `reset` that re-sources the deck (re-evaluating `.param` expressions and re-stamping device values). Per parameter a `kind` (OPT_ALTER / OPT_DECKPARAM) is tracked; when any `-dparam` is present `opt_eval` applies ALL deck params first, re-sources ONCE, THEN applies the in-place `alter` params (reset rebuilds from the deck and would otherwise wipe an earlier alter — this ordering is what lets the two kinds mix; verified on a joint `.param`+device fit). No `-dparam` ? the E-130/-143 fast path is untouched (no reset, no perf hit). The two re-source banners (`Reset re-loads circuit` … and `Circuit:` … in inp.c) are gated by the existing `ft_optimizing` flag so hundreds of inner re-sources are silent (only the final leave-at-optimum run shows); `ft_optimizing` re-asserted after each reset. Also fixed 5 pre-existing `-Wsign-conversion` warnings in inp.c's `*ng_script_with_params` handler (argc/size unsigned). Closes the last E-143 follow-up. No new files, no ABI change. Verified 31/31 (E-130/-143 [1]–[10] + E-144 [11]–[15]): scalar `.param` fit (rtop→2333.3), mixed `-dparam`+`-param` joint fit (rtop=3k,R2=2k), `.param` in expression (k→6), least-squares `-dparam` (LM), quiet re-source (≤1 banner); AC + OSDI-device-value `.param` verified manually

Enhancement	ngspice files	One line
E-145	frontend/com_optimize.c, frontend/commands.c, examples/optimize_examples/verify_optimize.py, examples/optimize_examples/optresm.va, examples/optimize_examples/optimize_mparam_demo.cir	**optimize** tunes **.model**-card parameters via a third knob kind -mparam. -param remains an alter instance target and -dparam a symbolic .param (re-source); -mparam <@model[param]> names a .model-card parameter (e.g. @dmod[is], @rmod[r]) — which is NOT alter-reachable and NOT .dc-sweepable (only sources/resistors/instance params are) — and is changed in place with altermod <name>=<value>, no re-source (as cheap per eval as -param, unlike -dparam). A new kind value OPT_MODELPARAM; in opt_eval the deck params are re-sourced first (E-144), then the in-place knobs: alter for instances, altermod for models. @model[param] is passed straight to altermod (which accepts that accessor form), mirroring how -param @m1[w] emits alter @m1[w]=.... Circuits with only -param/-mparam keep the E-130/-143 fast path (no reset). All three knob kinds mix in one run. Change confined to com_optimize.c (+1 help line); no new source files, no inp.c change, no ABI change. Verified 39/39 (E-130/-143/-144 [1]–[15] + E-145 [16]–[20]): OSDI model-param fit (@rmod[r]→3k), built-in diode model-param fit (@dmod[is]→1.22e-14), determined model+instance joint fit (r=3k,R2=2k), -mparam does 0 re-sources (in-place fast path), and all three kinds (-dparam+-mparam+-param) coexist and converge

<table>
<tr><th>Enhancement</th><th>ngspice files</th><th>One line</th></tr>
<tr><td>E-146</td><td>frontend/com_sweep.c (new), frontend/com_sweep.h (new), frontend/commands.c, frontend/com_commands.h, frontend/inp.c, frontend/Makefile.am, examples/sweep_examples/*</td><td>universal sweep command + .sweep card. Sweeps ANY circuit knob, generalizing .dc (which steps only sources/resistors/instance params). sw_kind auto-detects the knob: @X[y] with ft_sim->findModel(ckt,X) non-NULL ⍰ model param (altermod), else instance (alter); a bare name with nupa_get_param(name,&found) found ⍰ .param (alterparam+reset), else device (alter) — so model params and .params, impossible with .dc, are now sweepable. sweep <knob> (<start> <stop> <step> | lin|dec|oct N a b | list …) [-analysis <cmd>] [-output <name>=<expr> …]: sets the knob, runs the inner analysis (default op) at each point, evaluates each output (last value; default = all node voltages like .dc), and emits a summary plot named sweep with the knob values as the scale (nutmeg vector API, front-end layer per E-142). The per-point analysis plots are retained (tran1,tran2,…). .sweep card: inp.c collects a top-level .sweep line into the post-parse control list as a sweep command; a static sweep_active re-entrancy guard stops a .param re-source (reset re-runs the .sweep card) from recursing. New source files registered in commands.c/com_commands.h/Makefile.am. Verified 11/11 (verify_sweep.py): sweep R1 reproduces built-in .dc R1 bit-for-bit, model-param (@rmod[r]) + .param (rtop) sweeps vs analytic divider, auto-detection routing, .sweep card == command form (no recursion), AC named-output vs analytic |H(1kHz)|, transient settled value, lin/list/step point counts, multi-output</td></tr>
</table>

Enhancement	ngspice files	One line
E-149	maths/misc/randnumb.c, include/ngspice/randnumb.h, frontend/numparam/xpressn.c, frontend/numparam/spicenum.c, frontend/commands.c, examples/lhs_examples/*	Latin-Hypercube Monte Carlo sampling (`mcsample`), closing the low-discrepancy-sampling gap vs commercial tools. A stratified sampler in `randnumb.c` holds the mode/N/sample-index/dimension-counter/seed; each random dimension's stratum permutation (Fisher–Yates over `0..N-1`) + per-sample jitter are generated lazily from a self-contained `splitmix64` seeded by `(seed, dimension)` — reproducible and order-independent. `mc_sample_uniform()` = `(perm[d][i]+jitter[d][i])/N`; `mc_sample_gauss()` maps it through `inv_normal_cdf` (Acklam probit, rel err < 1.15e-9) so Gaussian tails stratify too. Sample boundary: `spicenum.c`'s `nupa_signal(NUPADECKCOPY)` edge (once per `reset` re-source, guarded by the existing `firstsignalS` latch) calls `mc_sample_advance()` (step sample, rewind dimension) before that pass's `.param` draws. Draw hook (`xpressn.c`): `agauss`/`gauss` take one `mc_sample_gauss()` and `aunif`/`unif`/`limit` one `2·mc_sample_uniform()-1` when LHS is active, else the unchanged `gauss1()`/`drand()` (LHS-off is byte-identical to before). Command `com_mcsample` (`randnumb.c`, registered in commands.c both tables) parses `lhs <N> [seed <s>]` / `random` / `off`. Front-end only, so solver-independent. Verified 5/5 under both solvers (verify_lhs.py): stratification (each of 48 strata hit once vs random's ~32/48), two-param multi-dimension, ~130x lower `Var(sample-mean)` at N=32 (both converge to the same mean), seed reproducibility, analytic mean/sigma correctness

Enhancement	ngspice files	One line
E-150	frontend/com_sweep.c, frontend/commands.c, maths/misc/randnumb.c, include/ngspice/randnumb.h, examples/_setup.py, examples/highsigma_examples/*	high-sigma rare-event estimation (`highsigma`), the last statistical ROW versus a commercial simulator -- 4-6 sigma failure probabilities (SRAM / standard-cell yield) that plain MC cannot reach (1e-7..1e-9 needs 1e7..1e9 runs). Scaled-sigma importance sampling: a new `MC_MODE_SSS` in the E-149 sampler draws each Gaussian `.param` from the lambda-inflated normal (`z = lambda*gauss1()`) and accumulates that draw's log likelihood-ratio `log(lambda) - (z^2/2)(1 - 1/lambda^2)` into `sss_logw`; `mc_sample_weight() = exp(sss_logw)` reweights the sample so the estimator is unbiased for the nominal probability. Uniform `.params` are bounded so SSS leaves them un-inflated (weight 1). Direction-free -- no gradient / sensitivity / most-probable-failure-point. `mc_sss_config(N,lambda,seed)` seeds the global PRNG for reproducibility; `mc_sss_off()` reverts. Command `com_highsigma` lives in com_sweep.c (reuses its `sw_run_cmd` reset/analysis runner and `sw_eval_expr`), registered in commands.c: `highsigma <N> [-scale <lambda>] [-seed <s>] [-analysis <cmd>] -metric <expr> [-max <hi>] [-min <lo>]` loops N samples (reset -> analysis -> eval metric -> spec test -> read weight), reports P(fail), relative error, equivalent one-sided sigma-to-fail (-Phi^-1(P)) and failure count, and leaves them in the `highsigma_*` vectors/vars. The spec is `-max`/`-min` values (not a >/< inside the expr, which a control command reads as an I/O redirect). Front-end only, solver-independent; the verify is heavy (thousands of re-sources) so `examples/_setup.py` marks `highsigma` SPARSE_ONLY. Verified vs analytic Phi(-beta) (verify_highsigma.py): beta=4 -> sigma 4.000, P 3.16e-5 (true 3.17e-5); deep tail beta=5 (P 2.87e-7) recovered from 6000 runs where plain MC expects ~0.0017 failures; two-sided spec ~doubles P; seed reproducibility exact; two independent Gaussians combine as N(.,sqrt(s1^2+s2^2))

Enhancement	ngspice files	One line
E-151	maths/misc/randnumb.c, include/ngspice/randnumb.h, frontend/numparam/xpressn.c, frontend/com_sweep.c, frontend/commands.c, examples/_setup.py, examples/yield_examples/*	process/mismatch correlations + packaged Monte Carlo yield -- closes the last two partial (warn) statistical rows vs a commercial simulator. (1) CORRELATIONS: `mc_corr_config(k, matrix)` Cholesky-factors a k x k correlation matrix (row-major, unit diagonal; rejects a non-positive-definite one) into lower-triangular `corr_L`; `mc_corr_component(i)` lazily draws the k underlying z's once per sample THROUGH `mc_sample_gauss()` (so LHS stratification / SSS weighting apply), computes y = L*z, caches it, and returns y[i]. The cache is reset in `mc_sample_advance()` which now runs its reset in EVERY mode (so correlation works under plain MC, not just LHS/SSS). New `.param` function `mvnorm(i)` (frontend/numparam/xpressn.c: added to fmathS, XFU_MVNORM enum in fmathS order, and the dispatch) returns the i-th correlated standard normal; with no matrix registered it degrades to an independent draw. `com_mccorr` (`mccorr <k> <m11>..<mkk>`
E-152	include/ngspice/optdefs.h, include/ngspice/tskdefs.h, include/ngspice/cktdefs.h, include/ngspice/smpdefs.h, spicelib/analysis/cktsopt.c, spicelib/analysis/cktntask.c, spicelib/analysis/cktdojob.c, maths/ni/niinit.c, maths/KLU/klusmp.c, examples/klu_tuning_examples/*	KLU matrix reordering + scaling controls. The KLU direct solver ran on the compiled-in `klu_defaults` (AMD ordering, `max` row scaling, BTF on) with no way to change them, and its one exposed knob `klu_memgrow_factor` was broken (`task->TSKkluMemGrowFactor = (val->rValue == 1.2)` stored a boolean, not the value). E-152 exposes three new `.options` -- ‘klu_ordering=amd
E-153	maths/ni/niiter.c, maths/KLU/klusmp.c, include/ngspice/smpdefs.h, include/ngspice/optdefs.h, include/ngspice/tskdefs.h, include/ngspice/cktdefs.h, spicelib/analysis/cktsopt.c, spicelib/analysis/cktntask.c, spicelib/analysis/cktdojob.c, examples/trustregion_examples/*	Levenberg-Marquardt trust-region Newton (`.option trustregion`, off by default), completing the damped/trust-region-Newton row alongside the E-111 line search. In the Newton loop (niiter.c): the E-111 KCL-residual merit is reused (gated on linesearch

| E-154 | frontend/com_envelope.c (new), frontend/com_envelope.h (new), frontend/commands.c, frontend/com_commands.h, frontend/Makefile.am, spicelib/analysis/envelope.c (new), spicelib/analysis/Makefile.am, include/ngspice/cktdefs.h | Envelope Following (`envelope <node> <fc> <tstop> [nppp] [m] [maxm] [reltol] [set` the last missing analysis in the RF/periodic-steady-state suite. For a carrier-driven circuit whose amplitude/phase envelope varies slowly over many carrier periods, it samples the state once per carrier period T=1/fc and integrates the slow drift, jumping M periods at a time. The exact per-period map

is X_{n+1}=phi(X_n), phi = one-period DAE integration; the envelope obeys dX/dn ~ phi(X)-X. The naive forward-Euler jump X_{n+M}=X_n+M(phi(X_n)-X_n) is UNSTABLE on high-Q circuits (the one-period monodromy has unit-circle eigenvalues, so I+M(Phi-I) amplifies -> blow-up; this is what shelved an earlier attempt). The new engine (spicelib/analysis/envelope.c, EFanalysis) uses the IMPLICIT backward-Euler jump X_{n+M}=X_n+M(phi(X_{n+M})-X_{n+M}), solved by Newton [(1+M)I - M*Phi] dY = -G with Phi=dphi/dY the one-period monodromy (finite-differenced, one extra period-integration per state, dense NxN solve capped for modest circuits). A-stable, so it tracks a resonator's envelope without diverging; its fixed point phi(X)=X is the true steady state. Step size M is chosen by step-doubling local-truncation-error control. The one-period map reuses the transient primitives (NIcomCof + NIiter per fixed sub-step, the dctran state rotation) on a fixed grid of nppp points in TRAPEZOIDAL mode (backward-Euler damps a high-Q resonance); phi is SELF-STARTING (a frozen-history variant plateaued at a false low fixed point) with a sub-divided backward-Euler first sub-step to bound the restart damping. The com_envelope.c front-end command runs a short settling transient, calls EFanalysis, and emits an `envelope` plot (_amp = 2|V1|, _dc, _re, _im vs time; nutmeg vector API). Solver-independent. Verified against a full `.tran` under both linear solvers: EF amplitude tracks the transient across a Q~3160 tank ring-up to <3% and to the steady state (<0.5%), stays bounded over 3000 carrier periods (26 envelope samples), and tracks a Q~316 tank to ~1.6%. Completes the RF suite |

| E-155 | frontend/com_reduce.c (new), frontend/com_reduce.h (new), frontend/commands.c, frontend/com_commands.h, frontend/Makefile.am, spicelib/analysis/rcreduce.c (new), spicelib/analysis/Makefile.am, include/ngspice/cktdefs.h, examples/reduce_examples/* | RC network reduction (`reduce <fmax> [factor f] [file fname] [name subckt] [keep node ...]`), moving the post-layout "RC reduction / model-order reduction" gap to done. Post-layout extraction produces enormous linear parasitic R/C networks (thousands-millions of interior nodes); `reduce` collapses the R/C network to a small ELECTRICALLY EQUIVALENT one preserving the port behaviour over DC..fmax, written as an ordinary `.subckt` of R's and C's. Method (spicelib/analysis/rcreduce.c, CKTreduceRC): TICER (Time-Constant Equilibration Reduction) -- Schur-complement (Gaussian) elimination of interior nodes of Y(s)=G+sC, kept first order in s so the reduced network is REALIZABLE as R's and C's (no model-order-reduction black box, no passive-synthesis step). Per elimination of node n, for each neighbour pair (a,b): G[a,b] -= G[a,n]G[n,b]/G[n,n]; C[a,b] -= (G[a,n]C[n,b]+C[a,n]G[n,b])/G[n,n] - G[a,n]G[n,b]C[n,n]/G[n,n]^2. The conductance update is the exact Schur complement so DC is preserved EXACTLY; the capacitance is matched to first order. A node is eliminated only when its self time-constant frequency f_n=G_n/(2*pi*C_n) > factor*fmax (quasi-static in the band of interest); `factor` trades reduction against in-band accuracy, monotonically. PORTS (kept nodes) are auto-detected as every node touched by a device that is NOT a resistor/capacitor (sources, transistors, OSDI Verilog-A devices), plus ground and user `keep` nodes, via the generic GENnode/terminal-count device interface -- so built-in and OSDI devices alike. The com_reduce.c front-end command enumerates the built-in resistor/capacitor instances (CKTtypelook), builds dense G/C over the RC nodes, runs TICER, and emits the reduced subckt (CKTnodName for node names). Solver-independent (reads devices + own dense solve). First cut is dense (node cap ~2500); sparse TICER is the follow-up. Verified under both linear solvers: a huge factor eliminates nothing and the emitted subckt reproduces the full AC response BIT-for-BIT; a moderate factor cuts nodes (e.g. 25->6, 4x) with <0.25 dB in-band error and DC exact; the accuracy/reduction tradeoff is monotone in factor; an OSDI device on a node auto-marks it a kept port |

| E-156 | spicelib/analysis/rcreduce.c, frontend/com_reduce.c, include/ngspice/cktdefs.h, examples/reduce_examples/* | Sparse RC reduction -- makes the E-155 `reduce` command scale to real post-layout size. The dense N*N TICER engine (capped ~2500 nodes) is replaced by a SPARSE one: the network is stored as per-node adjacency lists (RCadj: a growable edge list of {neighbour, g, c}), and interior nodes are eliminated in a MINIMUM-DEGREE order (a lazy binary heap keyed on current degree, like sparse LU) so fill-in stays tiny -- a degree-2 chain node merges two series elements with ZERO fill. A FILL GUARD (`maxdeg`, default 12) declines to eliminate a node once its degree grows past the threshold, so a dense mesh core (whose boundary Schur complement is inherently dense) is left intact instead of densifying*

*the whole network (measured: unguarded, one mesh elimination created 3308 fill edges; guarded, 9). The TICER Schur-complement update, DC-exact conductance, frequency criterion (eliminate only when f_n=G_n/(*2pi*C_n) > factor*fmax), and automatic port detection are unchanged from E-155; in the edge representation the diagonal is implicit so eliminating a node just adds Schur edges among its neighbours. Node cap raised 2500 -> 5,000,000; a 65,017-node network reduces in ~4 s. New maxdeg command option (com_reduce.c) and CKTreduceRC parameter (cktdefs.h). Also fixes a terminal-ordering pitfall: the reduced .subckt's terminals are emitted in internal-index order (not the order keep nodes were typed), so instantiating with the wrong order silently swaps ports -- the command now prints the correct x1 <ports...> <name> instantiation line. Verified under both linear solvers: identity reduction bit-exact (0.00 dB), a moderate factor gives ~4x fewer nodes at <0.25 dB in-band with DC exact, monotone accuracy/reduction tradeoff, OSDI auto-port, and an 8001-node network (past the old dense cap) reduces successfully | | E-157 | frontend/com_aging.c (new), frontend/com_aging.h (new), frontend/commands.c, frontend/com_commands.h, frontend/Makefile.am, examples/aging_examples/* | Device aging (aging <t_target> [rate <opvar>] [param <ageparam>] [dynamic <tstop> [tstep]] [verbose]), adding the reliability "stress -> degrade -> re-simulate (fresh/aged)" flow (HCI/NBTI/TDDB) and moving those reliability gap rows to done. The command finds every aging-capable device in the loaded circuit, computes how much it has degraded after t_target seconds of operation, writes that back into the device, and RE-STAMPS the circuit so any analysis run afterwards (op/dc/tran/ac) sees the aged devices; it runs the fresh stress simulation first, leaving it as the current plot (a "fresh" baseline). The design is MODEL-AGNOSTIC: a device opts in purely by exposing, in its Verilog-A/OSDI model, a degradation-RATE operating-point variable (default agerate, in dose-units/second) and a per-instance AGE parameter (default age, (*type=\"instance\"*)). The engine's only job is to integrate the rate into a dose and feed it back; the model owns the physics that maps age to a parameter shift (a sublinear power law, an Arrhenius factor, ...). Participants are detected by scanning each device TYPE's instance-parameter table (DEVices[t]->DEVpublic.instanceParms) for the two keywords, so ordinary resistors/sources are silently skipped and probing never errors. Two modes: STATIC (default) reads the rate at the DC operating point and scales by the lifetime, age = agerate(op)*t_target; DYNAMIC (dynamic <tstop>) runs a transient over one representative window, trapezoidally (time-weighted) integrates the rate, and extrapolates, age = (INT agerate dt / tstop)*t_target -- capturing duty cycle. Because age is per-instance, devices sharing a .model but at different bias age independently. Implementation (frontend/com_aging.c) reuses the com_optimize synchronous command-dispatch pattern to run op/tran, the nutmeg expression engine to read @inst[agerate], and alter @inst[age]=... to write back; console chatter suppressed via ft_optimizing unless verbose. Solver-independent. Verified under both linear solvers with a demo square-law NMOS carrying an NBTI hook (agemos.va): enumeration picks exactly the ageable devices, the dose is exactly rate*t_target, the threshold shift matches the analytic ΔVth = dvth_ref*(age/age_ref)^0.25 law to 5 sig figs, aged drain current drops, degradation is monotone in lifetime, a near-threshold device loses a larger fraction of its current than a hard-driven one for the same shift, a 30%-duty pulsed gate ages at 0.30x the DC rate (dynamic), and a sub-threshold device accrues zero dose | | E-158 | frontend/com_emir.c (new), frontend/com_emir.h (new), frontend/commands.c, frontend/com_commands.h, frontend/Makefile.am, examples/emir_examples/* | Power-grid EMIR (emir [rail V] [thresh frac] [thick m] [jmax A/m2] [n exp] [tref s] [top k] [verbose]), adding electromigration + IR-drop reliability sign-off and moving the last reliability gap row to done. After a DC solve of the power-distribution network the command reports two metrics. IR-DROP: for every node, the drop below the ideal rail (rail - V(node)); the rail defaults to the highest node voltage (the supply pad) unless given; reports the worst node and every node past a threshold (default 10% of the rail). ELECTROMIGRATION: for every wire-segment resistor, the current DENSITY J = |I|/(w*thickness), ranked; the worst-density segment, every segment past the current-density limit Jmax, and a Black's-equation relative lifetime MTTF/ref = (Jmax/J)^n (Black: MTTF ~ J^-n; a segment exactly at Jmax has MTTF = the reference lifetime tref, needing no process-specific prefactor). The physics the analysis exists to expose: EM is set by current DENSITY, not current -- a fat trunk carrying huge current can be safe while a thin wire at a fraction of that current voids first, which is why real grids taper wire width with carried current to hold J roughly constant. Implementation (frontend/com_emir.c) runs a fresh op (com_optimize synchronous-dispatch *pattern), walks the current plot's node-voltage vectors (plot_cur->pl_dvecs filtered to SV_VOLTAGE) for IR-drop, and enumerates the resistor instances (the device type named "Resistor") reading each segment's current and width via the nutmeg expression engine (@Rk[i], @Rk[w]) -- so it needs no device-struct access and works for any resistor; segments without a width are skipped and counted. Both tables are qsort-ranked (IR by drop, EM by density) and truncated to* `top`*. Solver-independent (reads a DC solution + per-resistor currents). Verified under both linear solvers on a 3-segment tapered ladder off a 1 V rail: worst IR drop 0.30 V (30%) at the far tap matching the exact ladder solve and linear in load current; J = I/(w*thick) exact; the worst-density segment is the narrow low-current wire, not the high-current wide trunk; the Black MTTF ratio between two segments equals (J2/J1)^n; with Jmax between the trunk and leaf density exactly the under-sized segments fail; rail auto-detect equals an explicit rail; and an OSDI (Verilog-A) current load at a tap is handled identically to a current source | | E-159 | (no ngspice source change) examples/compactmodels_examples/* | Real production compact-model bring-up -- a validation/example enhancement that exercises the EXISTING toolchain on actual CMC (Compact Model Coalition) Verilog-A models, so no ngspice or openvaf-r source change. BSIM4 (the ~12,600-line industry-standard bulk MOSFET) is compiled through openvaf-r to OSDI (~6 s) and validated against ngspice's BUILT-IN BSIM4 -- the same model, so a rigorous self-check: the OSDI BSIM4.8 and native BSIM4.8.3 agree to ~2% on the Id-Vds output family and ~4.5% on the Id-Vgs transfer curve (the residual is the point-release version gap). EKV (EPFL compact MOSFET, which ngspice has NO built-in for) is brought up purely through OSDI, giving textbook saturation. A general finding surfaced: OSDI keeps every internal node STATIC (no dynamic node collapsing), so BSIM4's internal drain/source nodes (di/si) -- which the default rdsmod=0 expects the simulator to collapse onto the external d/s -- are left floating and the device conducts ZERO current; enabling the external series-resistance nodes with rdsmod=1 (the model near-shorts them, 1e3 mho, when no S/D geometry is given) connects them and the device works. Models are compiled in place from the bundled OpenVAF integration-test sources (licenses stay with them). Verified under both linear solvers (6 checks) | | E-160 | (no ngspice source change) examples/cmcsweep_examples/, *examples/_setup.py | CMC compact-model coverage sweep -- the comprehensive follow-up to E-159, again a validation/example enhancement with no ngspice/openvaf-r source change (only _setup.py gains* `cmcsweep` *in its SPARSE_ONLY and REGRESSION_EXCLUDE sets so the slow 19-model compile runs single-solver and off the fast sweep). It drives EVERY real CMC (Compact Model Coalition) Verilog-A model bundled with OpenVAF through openvaf-r -> OSDI -> ngspice and reports a coverage matrix, the same idea as the E-84 LRM sweep but for production compact models. Result: 19 of 19 models COMPILE and 19 of 19 LOAD, spanning every device class -- MOSFET (BSIM3, BSIM4, BSIM6, BSIMBULK, EKV, HiSIM2, HiSIMSOTB, MVSG_CMC), FinFET (BSIM-CMG, BSIMIMG), SOI (BSIMSOI, HiSIMHV), GaN HEMT (ASMHEMT), surface-potential (PSP102, PSP103), bipolar/SiGe-HBT (HICUML2, MEXTRAM), and diode (DIODE, DIODE_CMC). Deeper conduction+physics validation for a representative model per class: OSDI BSIM4 and BSIM3 match ngspice's BUILT-IN BSIM4/BSIM3 to ~2%/~7%; EKV gives monotonic saturating I-V; HICUML2 shows bipolar current gain beta ~ 100 on a Gummel plot; DIODE_CMC turns on exponentially (x16000 over 0.2-1.0 V). Two findings: (1) the E-159 internal-node/rdsmod=1 issue is BSIM4-SPECIFIC, not universal -- BSIM3 handles the zero-resistance case fine and conducts out of the box (its apparent zero at Vgs=1 V is just a high ~1.7 V default threshold); (2) HiSIMHV (6 terminals d,g,s,b,sub,temp) needs cosubnode=1 to enable the substrate node, and with the default cosubnode=0 its $fatal correctly guards the node-count mismatch (confirming $fatal works through OSDI). Models compiled in place from the bundled integration-test sources (licenses stay put). Verified (7 checks); the compile/load tiers are solver-independent | | E-161 | (no ngspice source change) examples/dynmodels_examples/* | Dynamic (AC/RF) compact-model validation -- extends the E-159/160 DC validation into the dynamic domain, again a validation/example enhancement with no ngspice/openvaf-r source change. Where E-159/160 exercised DC I-V, this exercises the DYNAMIC behavior, which flows through a different code path: OSDI's REACTIVE (charge) Jacobian stamping and ngspice's `.ac` analysis, on the real production models (compiled in place from the bundled CMC sources). BSIM4 C-V: the gate capacitance Cgg(Vgs) is extracted from `.ac` as Im(I_gate)/omega and swept over gate bias; it rises from ~40 fF in subthreshold (overlap+fringe) to ~129 fF in inversion (approaching Cox*WL) -- the textbook MOSFET C-V -- and the OSDI-compiled BSIM4 matches ngspice's BUILT-IN BSIM4 to under 1% at every bias (a tighter check than the ~2% DC

I-V match, since Cgg is dominated by the version-independent oxide term). Cutoff frequency fT (the frequency where AC current gain |h21|=|I_out/I_in| falls to 1): OSDI BSIM4 fT ~ 3.5 GHz matches the built-in to ~1%; HICUML2 (SiGe HBT, which ngspice has no built-in for) has zero default transit time (t0=0) giving infinite fT, so a realistic dynamic parameter set (t0=10 ps, 1 fF junction caps) is supplied and the resulting fT lands right at the transit-time limit 1/(2*pi*t0) ~ 15.9 GHz and rises with collector current -- textbook charge-control behavior. Verified under BOTH the Sparse and KLU solvers (.ac is supported by both), 4 checks | | E-162 | frontend/inp.c, examples/hb_examples/verify_hb.py | **.hb** dot-card for harmonic balance -- gives the E-134 hb command netlist dot-card parity with the PSS family (.pss/.pac/.pnoise/.pxf/.psp/.sp are dot-cards, while the HB family hb/qpss/hbosc were control-block commands only). A top-level .hb <f0> <K> [points] [maxiter] card now runs single-tone harmonic balance straight from the deck (no .control block needed). Rather than duplicating the HB machinery as a separate analysis JOB, .hb reuses the deck->control mechanism Enhancement-146 introduced for .sweep: during deck loading (frontend/inp.c) a top-level .hb ... line is stripped of its leading . and appended to the post-parse control list, so it executes as hb ... (the com_hb command) once the circuit is built -- inheriting all of the E-134 engine (argument parsing, the E-121 conversion-matrix Jacobian, E-135 source-stepping continuation, solver independence). A boundary check (next char after .hb is whitespace or end-of-line) keeps it from swallowing any future .hb* card. Before this, .hb produced unimplemented dot command '.hb' -- the only .hb handler in ngspice was a disabled #ifdef WITH_HB upstream stub (dot_hb) that merely redirected to the PSS analysis, not the E-134 engine. Verified (examples/hb_examples/verify_hb.py, +2 checks): the .hb dot-card run in plain batch mode (no .control) converges and produces a harmonic-balance spectrum bit-for-bit identical to the hb command form on a junction-limited diode rectifier; it threads its optional [points]/[maxiter] arguments and coexists with a following .control block (deck order preserved); both Sparse and KLU give identical results. (Like .sweep, a bare command-style dot-card with no other analysis card prints a benign "no simulations run" batch notice even though HB ran.) | | E-163 | frontend/inp.c, examples/qpss_examples/verify_qpss.py, examples/phasenoise_examples/verify_phasenoise.py | **.qpss** / **.hbosc** / **.phasenoise** dot-cards -- completes the harmonic-balance family's netlist dot-card parity begun by E-162's .hb. Three more top-level command-style cards now run their analyses straight from the deck: .qpss <expr> <f1> <f2> [periods] [maxorder] (two-tone quasi-periodic steady state, E-133/136; add the hb keyword for the frequency-domain form), .hbosc <oscnode> <K> [fguess] [tstab] (autonomous harmonic balance for an oscillator, E-140; the deck needs a .ic to start the oscillation), and .phasenoise <fstart> <fstop> [points] (oscillator phase noise, E-140; run after .hbosc). Each reuses the same one-branch deck->control mechanism as .hb (E-162) / .sweep (E-146): in frontend/inp.c a top-level .qpss/.hbosc/.phasenoise line is stripped of its leading . and appended to the post-parse control list, executing as the corresponding command once the circuit is built -- inheriting the full E-133/136/140 engines and their solver independence. Because the bridge preserves deck order, .hbosc (which finds the oscillator PSS + frequency) and .phasenoise (which reads it) compose, so a complete oscillator phase-noise run needs no .control block. A per-card boundary check (the char after the name is whitespace or end-of-line) keeps the cards distinct -- in particular .hbosc is NOT matched by the .hb branch, whose own boundary check rejects the trailing osc. Verified: the .qpss dot-card produces a two-tone spectrum bit-for-bit identical to the qpss command (examples/qpss_examples/verify_qpss.py); .hbosc + .phasenoise in a deck reproduce the command form's oscillation frequency and phase-noise spectrum exactly with order preserved (examples/phasenoise_examples/verify_phasenoise.py); both under Sparse and KLU. No regression to .hb/.sweep. | | E-164 | (no ngspice source change) examples/rfpa_examples/* | Large-signal RF characterization of a real transistor -- a validation/example enhancement (no ngspice/openvaf-r source change) that closes the production-model validation loop (DC E-159, coverage E-160, small-signal E-161, now large-signal). It drives a common-emitter amplifier built from the bundled HICUM/L2 SiGe HBT (compiled in place, with the E-161 dynamic params t0=10ps + 1fF caps) hard and extracts the large-signal RF figures of merit via TRANSIENT + Fourier/FFT: AM-AM gain compression (the power gain expands 17.8->20.7 dB then compresses -- the exponential-transconductance signature of a bipolar -- defining P1dB), harmonic distortion (the third harmonic follows the textbook 3:1 slope, HD3 ~ A^3), and two-tone third-order intermodulation (the IM3 products 2f1-f2 / 2f2-f1 at ~-30 dBc).

IIP3, extracted from the single tone as A/sqrt(3*HD3) (the two-tone IM3 is exactly 3x the single-tone HD3 for a third-order nonlinearity), is constant at ~0.134 V across drive level, confirming the third-order model. A real finding: the frequency-domain harmonic-balance engines (hb/qpss, E-134/136, which just gained netlist dot-cards in E-162/163) do NOT converge on this stiff, many-internal-node production model in an amplifier configuration -- hb returns error 103 at any drive level (even 1 mV) and with extra iterations/source-stepping, and the two-tone qpss is prohibitively slow (>2 min per point). Transient+FFT integrates reliably and is the robust route for large-signal RF on a full production compact model; HB/QPSS remain the tool of choice for lighter, well-conditioned circuits. Verified under both linear solvers (4 checks): transient small-signal gain matches .ac (7.744 vs 7.742), HD3 3:1 slope (62.9 vs 64), IIP3 constant (0.134 V, spread <2%), and gain compresses at high drive (7.74->5.97). | | E-165 | (no ngspice source change) examples/modelnoise_examples/* | Production compact-model noise validation -- a validation/example enhancement (no ngspice/openvaf-r source change) exercising the last untested small-signal path, OSDI's `.noise` stamping of the models' own white_noise / flicker_noise sources, on production models compiled in place from the OpenVAF integration-test sources. BSIM4 (MOSFET): the output-noise spectral density Sv(f) of a common-source amplifier is compared to ngspice's BUILT-IN BSIM4 over the full band and matches to ~1.5% everywhere, covering both the low-frequency 1/f FLICKER region (amplitude density sqrt(Sv) ~ 1/sqrt(f)) and the flat high-frequency THERMAL floor -- a stringent check since the flicker coefficients, thermal-noise model, and their bias dependence all have to line up. HICUM/L2 (SiGe HBT): no ngspice built-in, so validated against physics -- its default noise is WHITE (shot noise on the junction currents + thermal noise on the parasitic resistances, no flicker by default), a flat spectrum in contrast to the MOSFET's 1/f rise; with a small source resistance so the intrinsic device noise dominates, the output-noise floor tracks the collector SHOT-noise line sqrt(2q*Ic*RC^2) across two decades of bias current (within 6% at high bias where shot dominates) and scales as sqrt(Ic). Verified under BOTH the Sparse and KLU solvers (`.noise` works under both since E-113 fixed the KLU adjoint solve), 5 checks: BSIM4 spectrum matches built-in <4%, BSIM4 1/f flicker slope (Sv(1Hz)/Sv(10Hz) ~ sqrt(10)), BSIM4 flat thermal floor, HICUM white noise, HICUM shot-noise tracking + sqrt(Ic) scaling. Completes the production-model validation loop (DC E-159, coverage E-160, small-signal E-161, large-signal E-164, noise E-165). | | E-169 | frontend/syntaxhl.c (new), include/ngspice/syntaxhl.h (new), frontend/commands.c, frontend/Makefile.am, main.c, examples/syntaxhl_examples/* | Interactive command-line syntax highlighting -- the interactive prompt now colors the command line AS IT IS TYPED. The command word is shown GREEN the moment it is a recognized command, RED when it can no longer become one, and left the terminal default while it is still a valid PREFIX being typed (so `plo` is neutral, `plot` green, `plt` red); numbers are yellow, "quoted strings" magenta, and -option flags cyan. The command word is classified exactly as the interpreter resolves it -- the same case-insensitive walk of the live command table cp_coms that docommand() uses, plus the control keywords (if/while/foreach/begin/end/...) -- so the color always agrees with whether the word will actually run. Implementation: a pure engine cp_highlight_line() (frontend/syntaxhl.c) tokenizes a line and wraps each token in ANSI SGR color escapes; live coloring overrides GNU readline's rl_redisplay_function (installed by cp_synhl_init() from main.c right after rl_outstream is set). The replacement redisplay redraws the colorized line only when the prompt plus line fit on one terminal row and otherwise defers to readline's own rl_redisplay(), so a wrapped line is never corrupted; the cursor is repositioned with a plain CR + CUF (cursor-forward) so mid-line editing is unaffected. Because that manual cursor positioning bypasses readline's internal position tracking, readline's own accept-line stops emitting the closing newline, so the Enter key is bound to a wrapper (cp_synhl_accept) that emits the newline itself when coloring is active before running the normal accept-line -- otherwise a command's output would begin on the input line (when coloring is off the wrapper is a no-op and readline handles the newline as usual). Gated three ways: it is on by default only at an interactive TTY (isatty(rl_outstream)), is disabled by `set no_syntax_highlight` and by the NO_COLOR environment convention, and never fires in batch (`ngspice -b`) or any piped/redirected session -- so simulation output is byte-for-byte unchanged (confirmed by the full example regression). Because as-you-type coloring needs the terminal in raw mode it requires a readline-enabled build, which the shipped binaries are (CI builds with --with-readline=yes and the committed binaries link libreadline). A new `synhl <command line>` command prints the colorized form of a line non-interactively -- useful for previewing and, since it needs no terminal, what makes the coloring engine regression-testable in batch. Verified (examples/syntaxhl_examples/verify_syntaxhl.py, 11 checks): the engine via `synhl` in batch (valid command green, unknown red, valid prefix neutral, numbers/strings/options colored, case-insensitive) and the live as-you-type behavior driven through a pseudo-terminal (green/red/neutral on the fly, NO_COLOR and a non-tty suppress all color); the live layer auto-skips on a build without readline. Solver-independent (front-end only). | | E-170 | frontend/syntaxhl.c, include/ngspice/syntaxhl.h, examples/syntaxhl_examples/* | Semantic syntax highlighting -- extends E-169 from lexical to SEMANTIC coloring of the interactive line: it now checks whether the SIGNALS and EXPRESSIONS typed actually exist and parse. (1) A signal reference v(node)/i(source)/@device[param] is looked up read-only in the current plot (vec_fromplot, no evaluation, no vector creation); it keeps the default color if it exists and is drawn RED if it does not. (2) Inside an otherwise-valid expression only the invalid signal reddens -- `print v(a)+v(zzz)` colors just v(zzz), including signals nested in functions (sqrt(v(a))-v(bad)). (3) A genuinely malformed expression argument of an expression command (plot/print/gnuplot/asciiplot) reddens as a whole, tested with the real parser ft_getpnames_from_string(FALSE) with its tree freed. (4) Error output is drawn RED at an interactive terminal: cp_err is wrapped in a colorizing stream (funopen on BSD/macOS, fopencookie on Linux) that prepends the red SGR to each write; cp_curerr is pointed at the wrapper too so ngspice's per-command stream reset cp_ioreset() keeps it in place instead of restoring and closing it. A half-typed expression is NOT treated as an error and stays neutral -- a signal whose parenthesis has not closed (v(bP), an expression with unbalanced parens (v(a)+v(b) or one ending in an operator (v(a)+). KEY FIX: the expression parser's error handler PPerror writes straight to stderr, bypassing the cp_err muting, so an early version spewed `syntax error in line segment ...` onto the prompt while typing; expr_parses() now also mutes fd 2 (dup2 /dev/null) across the parse and restores it. Signal-validity is scoped to the unambiguous v()/i()/@ forms (bare words could be plot keywords like vs, functions like sqrt, or options -> false reds). Same three-way gating as E-169 (interactive TTY, set no_syntax_highlight, NO_COLOR); the parser + vector lookups run per keystroke but are read-only and do NOT corrupt command execution (a subsequent let z=v(a)+v(b) still computes correctly). Verified (verify_syntaxhl.py, 19 checks total): the E-169 lexical checks plus a semantic layer driven through a pseudo-terminal after simulating a small circuit -- valid signal not red, invalid signal red, invalid-in-expression reddens only the signal, malformed expression red as a whole, half-typed stays neutral with no parser-error spam, error output red, NO_COLOR suppresses it. Solver-independent (front-end only). | | E-171 | maths/KLU/klusmp.c, spicelib/analysis/pzan.c, examples/klupz_examples/* | KLU/Sparse solver audit -- KLU pole-zero fixed (complex determinant + pivot tolerance) -- a deep audit of the KLU<->SMP bridge found that pole-zero analysis under 'option klu' SILENTLY PRODUCED GARBAGE for any circuit with complex poles or zeros: a series RLC reported four bogus real poles (-100, -10, -0.89, 0) instead of its conjugate pair -5000 +- j999987.5. Real-axis roots came out right (the mixed formula is accidentally correct for real pivots), which is why the existing single-real-pole RC regression never caught it. DEFECT 1 (spDeterminant_KLU): the complex determinant built each pivot as the mixed quantity (1/(Ux*Rs), Uz*Rs) and took its complex reciprocal -- KLU's Udiag stores the ACTUAL pivots (its solve divides by them) while Sparse's Diag stores RECIPROCAL pivots (its solve multiplies), and the adaptation kept Sparse's reciprocal dance for the real part only. The real branch was worse: its product loop NEVER RAN (the loop index was left at N by the preceding permutation scan, so det was always +/-1.0), it divided instead of multiplying, and it never wrote *piDeterminant (SMPcDProd consumed an uninitialized stack value). Both branches computed the permutation sign as #non-fixed-points/2, wrong for any cycle longer than 2 (a 3-cycle is EVEN but counted odd). Rewritten: det(A) = sign(P)*sign(Q)*prod(Udiag[k]*Rs[k]) with parity computed exactly by cycle decomposition (new PermutationParity helper); the KLU determinant now matches Sparse to ~14 digits at every Muller trial point. DEFECT 2 (SMPcReorder/SMPreorder): pole-zero calls SMPcReorder with PivRel = 0.0; Sparse's spOrderAndFactor SANITIZES a non-positive threshold to its stored default, but the KLU branch assigned it straight to Common->tol, making KLU's partial pivoting accept an EXACTLY-ZERO diagonal as pivot (|diag| >= 0*max always holds). At PZ's s=0 trial an inductor branch has a 0.0 diagonal (-sL), so the factorization returned KLU_SINGULAR and PZ recorded a spurious root at the origin -- and the poisoned deflated Muller search never expanded past |s| ~ 10, yielding the garbage above. Both reorder entry points now mirror Sparse's sanitization (in-range

PivRel still takes effect; DC/AC/tran pass the 1e-3 default and are byte-identical). With both root causes fixed, the E-113-era guard in pzan.c ('pole-zero finite-zero computation is not supported with option KLU' -- added when the zero search was seen to go spuriously singular, i.e. defect 2) is REMOVED; the finite-zero search now works under KLU and a genuine E_SHORT reports the same 'input shorted' diagnostic as Sparse. The balanced/differential-output PZ guard remains (SMPcAddCol genuinely has no KLU branch). Verified (examples/klupz_examples/verify_klupz.py, 6 checks): full pole/zero root-set parity between the two solvers plus analytic anchors on a series RLC (conjugate pair), RC lowpass (old-good case, no regression), lead network (finite real zero), RC highpass (origin zero), RLC bandstop (PURELY IMAGINARY zeros +-j1e6, the hardest case), and RLC bandpass (the finite-zero search formerly guarded off) -- all six identical across solvers. Full 134-example regression clean. Residual scope (solver-independent, ~1990 Muller-on-determinant fragility): a twin-T's deflated conjugate zero pair can still stall under KLU (4 of 6 roots found) while on the bandpass it is SPARSE that hits its iteration limit (KLU converges cleanly). | | E-172 | *maths/KLU/klusmp.c, spicelib/analysis/cktpzset.c, spicelib/analysis/pzan.c, examples/_setup.py, examples/klupz_examples/* | KLU balanced pole-zero + full partial pivoting -- closes E-171's two scope items; no analysis card remains Sparse-only under KLU. (1) BALANCED / DIFFERENTIAL-OUTPUT pole-zero ('pz in 0 a b vol', non-ground output reference) was guarded off as unsupported: the algorithm's change of variables needs the per-trial column fold col(b) += col(a) (SMPcAddCol), which had no KLU branch -- and KLU's CSC sparsity pattern is FIXED at COO->CSC conversion time, so the fold's fill-in cannot be created on the fly the way Sparse's spcCreateElement does. Fixed in two halves: CKTpzSetup reserves the UNION pattern before the conversion (every row of the solution column is also registered in the balance column via SMPmakeElt; duplicate (row,col) COO entries fold into a single CSC slot by the conversion's group labeling, so the reservation is safe), and SMPcAddCol gains a KLU branch that merge-walks the two columns' sorted row lists adding the complex values in place (with a defensive error if an addend row is missing, impossible after the reservation). The two pzan.c balanced-output guards are removed. (2) FULL PARTIAL PIVOTING for PZ factorizations: validating a fully-differential configuration exposed spurious far-field roots at |s| ~ 1e19..1e21 (including a positive-real pole). An exact-rational (fraction-arithmetic) determinant of the very matrix KLU factored proved the factorization itself was numerically rotten at extreme |s|: at s=4.52e19 the exact determinant was +4.36e11 while the pivot product gave -2.05e12 -- wrong sign AND magnitude. Root cause is architectural: KLU's ordering is fixed at klu_analyze time (pattern-only AMD/BTF) while Sparse re-runs value-aware Markowitz ordering every PZ trial; PZ sweeps |s| across ~20 decades, and with the relaxed default tol=0.001 the fixed ordering accepted catastrophically-cancelling pivots. KLU's only value-adaptive lever is the within-block partial pivoting, so the E-171 sanitization fallback is upgraded from 'keep current tolerance' to FULL partial pivoting (tol=1.0) whenever the caller passes an out-of-range PivRel -- only pole-zero does (0.0); explicit in-range tolerances (DC/AC/tran 1e-3) are honored unchanged. This single change eliminated the far-field junk AND cured the twin-T conjugate-zero-pair stall recorded in E-171's scope (what looked like vintage-Muller noise-floor fragility under KLU was largely this factorization inaccuracy; KLU now finds all 6 twin-T roots with notch-zero real parts ~1e-12, an order CLOSER to the exact 0 than Sparse's ~1e-11). Verified (verify_klupz.py, 6 -> 9 checks): the E-171 six plus the differential RC bridge (balanced output, formerly 'not supported'), a balanced output with a complex pole pair, and the twin-T notch -- all root sets identical across solvers. The dual-solver harness's balanced-pz KLU-skip detection is removed from examples/_setup.py (nothing remains Sparse-only). Full example regression 134/134. | | E-173 | spicelib/analysis/cktpzeig.c (new), maths/dense/eig.c (new), maths/KLU/klusmp.c, spicelib/analysis/pzan.c, cktsopt.c, cktntask.c, cktdojob.c, include/ngspice/{smpdefs,optdefs,tskdefs,cktdefs}.h, two Makefile.am, examples/pzeig_examples/ | *Eigenvalue-based pole-zero ('.options pzeig') -- a modern alternative root finder for .pz. The vintage driver hunts roots of det(G+sC) with a Muller iteration on determinant values and is famously fragile (iteration limits -- 'giving up after 231 trials' on a plain RLC bandpass -- noise-floor stalls, chaotic search paths; E-171/172 fixed the KLU-side defects but the ~1990 algorithm itself stays delicate under BOTH solvers). The new opt-in method computes every pole and zero at once as a dense eigenvalue problem, reusing the existing CKTpzSetup/CKTpzLoad machinery so it applies unchanged to the poles and zeros configurations, to balanced/differential outputs, and to both linear solvers. Steps: (1) the PZ-configured MNA matrix is AFFINE in s, $A(s) = G + s$*C, so two loads (s=0, s=1) extract the dense pencil via a new solver-agnostic SMPdenseExtractReal (KLU: walk Ap/Ai/AxComplex; Sparse: walk FirstInCol through the Int->Ext translation maps); a third load at an arbitrary point VERIFIES affinity so a non-polynomial device falls back with a clear message instead of wrong roots. (2) C is structurally singular, so the pencil is solved by SHIFT-INVERT linearization: factor (G + sigma*C) ONCE at a non-root shift (tried from a small ladder of shifts, with the circuit's own sparse solver), form the dense M = (G+sigma*C)^{-1}C by n sparse solves; from (G+sC)v=0 <=> Mv = mu*v with mu = 1/(sigma - s), every finite root is s = sigma - 1/mu and the pencil's infinite eigenvalues land harmlessly at mu = 0 (dropped by a noise-floor threshold 64*neps||M||). (3) eigenvalues of the real dense M from a new self-contained eigensolver maths/dense/eig.c -- the classical balance / Hessenberg / Francis double-shift QR chain (EISPACK balanc/elmhes/hqr lineage, written fresh, no LAPACK dependency), unit-tested standalone against companion matrices, complex conjugate pairs, a 50x50 tridiagonal with known spectrum, and a badly-scaled balance case. Roots are emitted as the same PZtrial list the Muller driver produces, so PZpost and all downstream output are unchanged. Option plumbing follows the standard task-option path (OPT_PZEIG -> TSKpzEig -> CKTpzEig flag -> dispatch in pzan.c). RESULTS: RLC bandpass -- same roots as Muller with NO iteration-limit warning; 10-section RC ladder -- all ten poles matching the analytic tridiagonal-eigenvalue formula s_k = -(2-2cos((2k-1)pi/21))/RC and Muller root-for-root; twin-T -- all 6 roots under both solvers; RLC bandstop -- purely imaginary zeros +-j1e6 EXACT; balanced/differential output works; purely resistive circuits yield no roots and no crash. DEFAULT REMAINS MULLER (pzeig is opt-in). Dense O(n^2) memory / O(n^3) QR, capped at 2000 unknowns with a clear error above. Accuracy is dense-QR class (absolute error ~ eps * dominant root magnitude): an exact origin-zero can print as ~1e-7 when poles sit at 1e6 rad/s; Muller refines locally and can resolve wider per-root dynamic range, while pzeig never misses or invents a root -- the two methods agree to >= 6 digits on every battery circuit. Verified (verify_pzeig.py, 13 checks: both solvers x both methods + analytic anchors + default-method check). Full example regression 136/136. | | E-174 | src/frontend/commands.c, examples/helpcmd_examples/* | **`help all`** crash fix (help string used as printf format) -- running `help all` (or `help montecarlo`) in the interactive shell crashed the shipped ngspice with 'Error: tvprintf failed / ERROR: fatal error in ngspice, exit(-1)'. Root cause: ngspice's help printer uses each command's help TEXT as a printf FORMAT string -- out_printf(ccc[i]->co_help, cp_program) in com_help.c (and the parallel com_ahelp.c) -- passing only one argument (cp_program, the program name, intended for a single %s as in the legitimate 'Report a %s bug.'). Any other '%' in a help string is an invalid/incomplete conversion specifier, so tvprintf fails and ngspice calls a fatal exit(-1). The montecarlo command's help (added in E-151) read '... reports the yield with a Wilson 95% CI ...'; the '% ' (percent-space) is the invalid specifier. help all walks the command table alphabetically and died at montecarlo; help montecarlo crashed directly too. Present in BOTH command tables (spcp_coms for ngspice, nutcp_coms for nutmeg). Not X11/terminal/solver specific -- plain format-string handling, so it reproduced on every build; it was invisible in headless CI only because help all is an interactive command not exercised by the batch example decks. Fix: escape the literal percent as %% (printf convention) in both tables, so the line renders 'Wilson 95% CI' and help all completes. New regression guard examples/helpcmd_examples/verify_helpcmd.py with two layers: RUNTIME -- drives an interactive ngspice on a pseudo-terminal and asserts help, help all, and help montecarlo each run to completion (no crash, no 'tvprintf failed') and that the montecarlo line renders a literal '95% CI'; and a STATIC class-guard that scans commands.c and fails if any command help string carries a format hazard (a '%' that is not '%s' or '%%', or more than one '%s' since only one argument is passed), catching a future unescaped '%' even when that command is never exercised at runtime. Full example regression 137/137. | | E-175 | spicelib/analysis/dcpss.c, examples/rfconv_examples/* | RF-suite audit -- the dropped parametric term in every periodic small-signal analysis -- a full audit of the RF suite (.sp/Touchstone, PSS/PAC/pnoise/pxf, PSP, HB, QPSS + the two-tone family, hbosc/phasenoise) driven by analytic anchors and cross-analysis invariants. CLEAN: .sp + Touchstone (17 checks: series-R/shunt-C/3-port star S-parameters exact to 10 digits, the asymmetric-2-port S21/S12 column-order trap, reciprocity, RI/MA/DB + Y/Z (v1 normalization) + GHz round-trips through wrsnp/rdsnp, N-port 4-pairs-per-line layout); HB (analytic cubic: fundamental a1*A+(3/4)*a3*A^3 and H3=(1/4)*a3*A^3 exact to 7 digits, even harmonics at machine zero, Sparse/KLU identical); QPSS-HB/QPSS-tran (two-tone fundamentals, IM3 (3/4)*a3*A^3 exact, commensurate aliasing guard fires correctly). THE BUG: every

LPTV small-signal conversion matrix scaled its reactive coupling by the COLUMN (input-sideband) frequency, H_nm = G_{n-m} + j*omega_m*C_{n-m}. For a small signal riding a FROZEN periodic capacitance C(t)=dQ/dv|orbit the exact linearization is di = d/dt[C(t)dv] = Cdot*dv* + *C*dv_dot, and the product rule collapses to the ROW frequency: H_nm = G_{n-m} + j*omega_n*C_{n-m}. Using omega_m silently DROPS THE PARAMETRIC-PUMPING TERM Cdot*dv -- the very term that makes a pumped varactor convert. Every conversion sideband through a time-varying capacitance came out scaled by exactly omega_in/omega_out: on the audit circuit (1k -> OSDI varactor C(v)=c0(1+alpha*v) pumped 1V@1MHz, 1mV probe at 250kHz) the +-1/+-2 sidebands were 3x/5x/7x/9x too small, with measured pre-fix/truth ratios matching the omega ratios to FIVE DIGITS (0.3333/0.2000/0.14286/0.11111) -- diagnosis confirmed beyond doubt. AFFECTED: pac, psp, pnoise, pxf, qpac, qpnoise, qpxf, and the phasenoise adjoint, for any circuit with pumped/nonlinear capacitance (varactors, junction and MOS capacitances -- every real mixer and oscillator). UNAFFECTED: LTI circuits (no off-diagonal C harmonics; omega_n == omega_m on the diagonal) -- exactly what every prior regression check used; the E-171 accidental-correctness pattern again. THE SUBTLETY: the harmonic-balance residual uses the SAME builders and must KEEP the column frequency -- its reactive current is the exact chain rule d/dt Q(v(t)) = C(v(t))*vdot(t) whose n-th harmonic is sum_m C_{n-m}*(j*omega_m*V_m). That is why HB and QPSS-HB always matched transient ground truth while the small-signal analyses did not, and why a blanket fix would have broken HB. FIX: a smallsig mode flag on pac_build_matrix and qp_build_matrix plus the two inline copies (the PAC adjoint and the phasenoise adjoint): smallsig=1 -> row frequency (restores the parametric term); smallsig=0 (HB/HBOSC/QPSS-HB residual + Jacobian) -> chain-rule column frequency, byte-for-byte unchanged. Regression safety proven directly: the linear-circuit PAC control produces IDENTICAL output pre/post fix, and HB/QPSS-HB outputs are byte-identical. Ground truth = plain transient + one-beat Fourier projection (independent of all conversion-matrix machinery); the fixed qpac matches all five sidebands within 2% (integrator error class). Also noted: cktspnoise.c is a dead fossil (#ifdef RFSPICE_ never compiled). Verified (examples/rfconv_examples/verify_rfconv.py, 5 checks x both solvers: truth vs QPSS-HB unchanged, truth vs fixed qpac, pre-fix signature absent, HB + QPSS-HB analytic anchors). Full example regression 138/138. | | E-176 | spicelib/analysis/dcpss.c, examples/_setup.py, examples/pssdriven_examples/* | Driven-mode PSS shooting -- no frequency hunt, no breakpoint flood -- diagnosis of the E-175 audit's flagged residual ('PSS shooting robustness: the varcap PSS ran 17+ minutes without converging'). Instrumentation showed 9,627,161 accepted timepoints by t=2us (~0.2 ps per step, zero LTE rejections): ngspice's PSS (the Lannutti shooting) is OSCILLATOR-oriented -- it hunts the fundamental frequency through mid-period forced breakpoints whose spacing is proportional to steady_coeff. On driven circuits (everything the E-117..126 periodic small-signal stack runs on) this is doubly pathological: (1) at the standard decks' steady_coeff=5e-6 the grid spacing is ~0.5 ps -- millions of steps per shooting cycle, and the convergence-tolerance knob doubles as grid density so tighter tolerance quadratically explodes runtime (even the default 1e-3 forces thousands of points/period -- why a linear RC .pss took ~4 minutes); (2) the frequency estimate can never settle EXACTLY on the source frequency, the cycle endpoint phase-drifts against the source, and the period residual FLOORS (~1e-4) -- shooting never converges and burns the full sc_iter budget at flood speed. FIX: a DRIVEN mode, auto-detected when the circuit contains a time-varying independent source (SIN/PULSE/... V or I source, funcTGiven): the period is pinned to the exact source period (no estimation, no estimator grid; each cycle is one plain-transient period; varactor residual 2e-1 -> 8e-9 in 17 cycles / 0.3 s); the shooting-phase max step is clamped to T/psspoints so the orbit is integrated on the SAME discretization as the retained samples (validation caught this: without the clamp LTE grows steps toward T/2 and shooting converges -- to 7e-9! -- onto the fixed point of that coarse discretization, several percent off the true orbit: retained swing 0.1651 vs analytic 0.15718; the old flood had been masking the effect by brute force); the post-FFT 'relaunch at the strongest spectral line' is disabled when driven (a rectifier-like circuit with a dominant 2nd harmonic must not retain the wrong period); the AUTONOMOUS oscillator path is untouched (byte-identical behavior verified on oscillator decks; DC-only-supply circuits never trigger detection). IMPACT: rc_pss ~4 min -> 0.05 s with the retained fundamental EXACTLY 1000000 Hz (the old hunt returned 999999.8976); psp example 406 s -> 0.9 s; the rfpss battery (5 verifies, 61 checks) minutes-each -> <0.5 s each; rfanalyses 'several minutes even Sparse-only' -> 0.56 s; the KLU PSS pass 'prohibitively slow' -> 0.14 s. HARNESS: rfpss and rfanalyses removed from both REGRESSION_EXCLUDE and SPARSE_ONLY -- the whole RF periodic small-signal suite now runs in EVERY regression sweep under BOTH solvers; the direct .pac-on-pumped-varactor check became affordable, closing the E-175 loop on the direct PAC path (sidebands match transient ground truth within 1%). Verified (examples/pssdriven_examples/verify_pssdriven.py, 6 checks x both solvers: driven detection + exact retained frequency, retained swing == |H| within 0.1%, retained-period self-consistency, direct .pac vs transient truth, autonomous non-trigger). Full example regression 145/145. | | E-177 | spicelib/analysis/dcpss.c, examples/pnoisefold_examples/* | pnoise noise-folding referee + the folded-flicker frequency fix -- closes the E-175 audit's last honesty-ledger gap: periodic noise analysis (pnoise/qpnoise/phasenoise) was 'correct by construction, not correct by measurement' -- the folding of device noise through LPTV conversion had never been checked against anything independent. THE REFEREE (three independent layers, on a 1-node LPTV-conductance circuit G(t) = g0 + g1*sin(w0 t)* chosen so conversion transfers are O(1)): (1) a from-scratch Python conversion matrix, onoise(f) = sum_k |Z_k(f)|^2 * S(|f + k*f0|) -- same physics, zero shared code; (2) a TRNOISE transient Monte-Carlo (no shared code OR theory; 198-segment Welch PSD, with the pumped/unpumped RATIO cancelling the TRNOISE amplitude convention); (3) the LTI (pnoise == .noise) and white limits. VERDICT ON THE WHITE PATH: measured-correct -- pnoise matched the referee to 6 digits at every frequency and the MC arbiter confirmed the ~1.86x folding ratio within statistical error; the adjoint, sideband sum and thermal densities are right. THE BUG THE REFEREE CAUGHT: the stationary sideband loops set the noise-evaluation frequency ONCE, to the output frequency (data.freq = freq outside the k loop). But the sideband-k adjoint carries noise ORIGINATING at |f + k*f0| -- a frequency-dependent source PSD (flicker 1/f, noise_table E-109) must be evaluated at the source-side frequency. The proof was digit-exact in both directions: pre-fix pnoise reproduced the referee's deliberately-wrong evaluate-at-f model DIGIT-FOR-DIGIT (1.114323e-14 vs 1.114323e-14 at 1 kHz -- a 21% overestimate on this circuit, unbounded as f « f0); post-fix it reproduces the correct model digit-for-digit (9.175472e-15). Why three generations of checks missed it: white noise is frequency-flat and LTI circuits have no k != 0 transfer -- the THIRD occurrence of the accidental-correctness pattern (E-171 determinant, E-175 parametric term). FIXED in all three stationary loops: pnoise (data.freq = |freq + k*f0| per sideband), qpnoise (|f_in + k1*f1 + k2*f2| per harmonic), and phasenoise -- where it matters most: folded far-sideband blocks near a carrier were evaluated at the tiny OFFSET frequency, inflating the flicker (1/f^3) region. CYCLO SCOPE: the cyclo mode's time-domain identity onoise = (1/P) sum_s S(t_s)|A_s|^2 treats the source PSD as frequency-flat across the folding span -- exact for modulated-white noise and for flicker on conversion-free circuits (the shipped E-125/126 use cases), not for flicker folded through k != 0 conversion (cannot be collapsed into that product with the aggregate device-noise API); documented at both cyclo sites, with the (now exact) stationary mode as the right tool when folded flicker matters. Verified (examples/pnoisefold_examples/verify_pnoisefold.py, 5 checks x both solvers, ~2 s thanks to E-176): white folding vs referee (<=0.1%), flicker folding vs referee (<=0.5%, sample-0 bias read from the retained orbit), pre-fix signature absent, pump/no-pump ratio == referee ratio, LTI limit == .noise. The rfpss/qpss/rccyclo suites pass unchanged (conversion-free or white -- provably unaffected). Full example regression 146/146. | | E-178 | spicelib/analysis/dcpss.c, spicelib/devices/dio/dionoise.c, examples/cyclofold_examples/*, examples/rfpss_examples/*, examples/qpss_examples/* | exact separable cyclostationary folding + the HB DC-source fix -- removes the cyclo mode's frequency-flat limitation E-177 documented. For the separable model S(t,f) = m(t)^2*g(f), onoise(f) = sum_g sum_q |B_q(g)|^2 * g_g(|f + q*f0|) with B_q(g) = (1/P) sum_s m_g(t_s)*dA_s(g)*e^{+jq theta_s} and A_s = sum_k Psi_k e^{-jk theta_s}. Per-generator amplitudes recovered with NO device-API change: the noise-summary machinery (prtSummary/outpVector, one density per generator; family totals excluded by the name-prefix rule) plus load POLARIZATION against a fixed reference load R = Psi_0 -- five DEVnoise sweeps per orbit sample (A+-R, A+-jR, R) give c_g,s = sqrt(S_g(t_s))*dA_s(g) up to a constant phase that cancels in |B_q|^2. Spectral shapes are MEASURED pointwise at the folded frequencies (no 1/f^EF assumption -- noise_table works) at several probe biases; flat generators keep the original time-domain identity (exact for ANY quadratic form incl. NevalSrc2-correlated pairs); stationary limit == the E-177 sum, flat limit == the old identity (Parseval). Ported 2-D to `qpnoise ... cyclo` (B_{q1,q2} at |f + q1 f1 + q2 f2|). THE PHYSICS: 'flicker sees ^2, white sees <m^2>' -- only the envelope's DC feeds the 1/f band; the old identity was 23% high (8/pi^2) on |sin|-modulated flicker and 34% high on the conversion circuit; the new sweep lands on a from-scratch referee to <=3e-4 (the old binary reproduced the deliberately-flat model to 6e-5 -- the error was the model, not the code). BONUS BUG (4th accidental-correctness occurrence): the cross-machinery check (same circuit through the QPSS-HB orbit) came out exactly 4x -- BOTH HB Newtons (`hb` E-134, `qpss ... hb` E-136) double-subtracted the DC sources (the settle-mode rhs folded into I_R already carries them; -lambda*Is subtracts them again): every DC bias voltage converged to exactly 2x, silently corrupting all bias-dependent noise and conversion (flicker ~I^AF off by 2^AF, shot 2x) while AC content and every AC-driven validation stayed exact. Fixed by adding the unscaled DC source block back (net DC drive exactly -lambda*Is_DC); `qpnoise cyclo` now agrees with `pnoise cyclo` DIGIT-FOR-DIGIT across the two orbit machineries. rccyclo's flicker law updated to the exact <|I|>^2 = (8/pi^2)<I^2> (pinned to 1e-4). 6 checks x both solvers; regression 147/147. FOLLOW-UP (same enhancement): the E-139 hard-pumped-diode deck exploded the cyclo path (~6.5e1 V^2/Hz) -- dionoise.c computes its three sidewall generators only when DIOresistSWGiven but the prtSummary block copies ALL DIONSRCS slots, so without a sidewall model the summary vector carried uninitialized stack garbage (latent in stock ngspice for decades; E-178 is the first consumer of per-generator densities). Fixed at the device (unwritten slots zeroed, dio/dionoise.c) and hardened in both cyclo consumers (finite non-negative PSD guards; garbage probe ratios fall back to the flat path). verify_qpnoise's 'cyclo > 2x stationary' diode expectation was an artifact of the garbage slots + doubled HB DC bias; replaced with a closed-form torus-average referee (resistive network => memoryless LPTV, instantaneous adjoints) matched to ~1%. | | E-179 | spicelib/analysis/tfanal.c, spicelib/analysis/cktsens.c, frontend/com_measure2.c, examples/stdaudit_examples/* | standard-analyses audit: referee battery + three fixes. The 'Standard analyses (analog)' gap-table rows are 1990s SPICE3 code whose VALUES had never been physics-checked. FIX 1 (tfanal.c, inherited from Berkeley SPICE3 1988): `.tf i(vm) vin` output impedance was ALWAYS 1e20 -- the output-impedance solve forces 1V on the sense branch and inverts the branch current as 1/MAX(1e-20, rhs), but that current is NEGATIVE for every passive network (the input-impedance path right above divides by -rhs); fixed with the same sign + fabs guard; feedback-amp referee exact (1000.990 == RL + node Thevenin). FIX 2 (cktsens.c): KLU AC sensitivity silently truncated to ONE frequency point -- the E-114 KLU complex-conversion block inside the frequency loop reused `i` (the OUTER loop variable) for its DEVmaxnum walk, so after the first point i=DEVmaxnum ended the sweep; the surviving point was correct, which is why E-62's single-point check passed. Fixed with a local index (dead second block hardened too); full sweep == analytic dV/dC = -jwR/(1+jwRC)^2 to 6 digits, both solvers. FIX 3 (com_measure2.c): `.meas DERIV{ATIVE}` was parsed but evaluation fell into an explicit 'currently not supported' stub (empty #if 0 measure_deriv() placeholder) -- implemented: 3-point Lagrange-quadratic derivative on the nonuniform time grid, AT= and WHEN forms sharing FIND's parse/locate flow, verified vs the analytic sine slope to 5 digits; DERIVATIVE/INTEGRAL long spellings accepted (only DERIV/INTEG matched); pre-existing -Wmissing-prototypes warning silenced (str_has_arith_char2 static). MEASURED-CORRECT: .disto HD2/HD3 == analytic diode Volterra kernels <=0.06% incl. a frequency-dependent load (harmonics at Z(2w)/Z(3w) -- no E-177-style frequency bug), SIM2 intermods (f1+f2, f1-f2, 2f1-f2) incl. cascades, exact DISTOF1/DISTOF2 scaling, junction-cap harmonics == Harmonic Balance to ~6 digits (two independent engines); .noise onoise_total^2 == band analytic (kT/C) 0.06% + flicker log-integral 6 digits; .sens DC == central finite differences at a nonlinear OP (5-6 digits, model params incl.); nonlinear-OP .pz works via the driving-point form (the E-62 'quirk' was a bias source shorting the injection node -- ngspice's refusal is correct); tran/op/meas battery exact. 8 checks x both solvers; regression 148/148. | | E-180 | frontend/com_checkpoint.c, examples/checkpoint_examples/* | checkpoint/restart under KLU (the last Sparse-only feature falls). E-131 guarded `savestate/loadstate` off under KLU, recording the reason as 'KLU factorization objects absent on the restore path' -- a mis-diagnosis. REAL cause: `.option klu` lives in the task (TSKkluMODE) and reaches the circuit only in CKTdoJob at analysis dispatch, but loadstate calls CKTsetup DIRECTLY first, so the matrix was built Sparse (SMPkluMatrix NULL); the resume dispatch then set ckt->CKTkluMODE=1 without re-setup (by design -- it must preserve the restored state), and the first NIiter dereferenced the NULL SMPkluMatrix (KLUloadDiagGmin store; reproduced EXC_BAD_ACCESS +0x90). savestate was never broken (the checkpoint file holds only solver-agnostic vectors). FIX: com_loadstate copies the task's solver selection and the E-152 KLU knobs (ordering/scale/BTF/memgrow) into the circuit

BEFORE CKTsetup, exactly CKTdoJob's block; both guards removed. BONUS: cross-solver restore -- all four save x load combinations (fresh processes) continue exactly on the uninterrupted trajectory (<=1e-9 at t=3.5ms of a 4ms diode+RC run). verify_checkpoint's 'KLU rejected' check replaced by the 4-combo matrix (22/22); solver-notes and gap-analysis tables updated. Nothing in the simulator is Sparse-only any more (E-172 closed the last analysis, E-180 the last feature). Regression 148/148. | | E-181 | spicelib/analysis/dctran.c, spicelib/analysis/cktsopt.c, spicelib/analysis/cktntask.c, spicelib/analysis/cktdojob.c, include/ngspice/cktdefs.h, include/ngspice/tskdefs.h, include/ngspice/optdefs.h, examples/corenum_examples/* | core-numerics audit: the integrator certified exactly + **`.options ordfix`**. Nobody had ever verified that the Gear (BDF) corrector coefficients deliver their nominal order (orders 3-6 were dead code for ~30 years until E-128 selected them; E-128 verified order SELECTION, not coefficient CORRECTNESS). NEW OPTION `ordfix=K` (off by default): a fixed-order verification mode -- every converged step is accepted (the step is ruled by tmax, not the LTE), the first step is shrunk 1000x (an order-1 starter proportional to the pinned step imposes an O(h^2) floor), the order ramps to K while the step is tiny, then the step grows gently (x1.15; large ratios destabilize high-order variable-step BDF). Each guard defeats a measurement trap that initially presented as a bug but is real numerics (incl. the LTE step preference h_k=(c_k*tol)^{1/(k+1)} INVERTING at loose tolerance -- the E-128 controller's refusal to raise there is correct). CERTIFIED: accepted trajectories satisfy the exact variable-step BDF-k formula (Lagrange differentiation on the actual nodes) to <=1.3e-13 at every order 1-6 (~90 uniform stencils each); measured global slopes confirm orders 1-3; trap-vs-Gear dissipation on a lossless LC matches theory (|R(iy)|=1 vs order-dependent damping). Also referee'd with NO defects: solver precision == conditioning theory under both solvers (1e18-conductance-spread asymmetric mesh == exact-rational MNA to 1e-15); all four DC convergence-aid paths (default/noopiter/gminsteps=0/srcsteps=0) agree on a 40-diode hard-DC chain (spread 2.2e-6, KLU==Sparse 7e-13); xmu=0.5 == trap bit-identical, xmu=0.45 damps; lvltim=1 works. Drive-by: DCtran_step_quit prototype added to cktdefs.h (pre-existing -Wmissing-prototypes). 8 checks x both solvers; regression 149/149. | | E-182 | frontend/plotting/plotit.c, frontend/plotting/pyplot.c, examples/pyplot_examples/* | pyplot autoscale by default -- the matplotlib bridge (E-94/95/98/99) pinned every generated axis with set_xlim/set_ylim taken from ngspice's grid-rounded internal plot ranges. User request: rely on matplotlib's autoscaling + fig.tight_layout() instead. plotit() always fills xlims[]/ylims[] (user args OR data-derived), and the statics that distinguish the user case (xlim/ylim from getlims) are freed BEFORE the device dispatch -- plotit now captures user_xlim/user_ylim flags at the free point and passes NULL limits to ft_pyplot unless the user gave xlimit/ylimit explicitly (ft_pyplot already handled NULL; gnuplot/asciiplot back-ends untouched). verify_pyplot +2 checks (no pins + tight_layout on auto plots; exact pins for explicit limits). Regression 149/149. | | E-183 | frontend/com_pyplot.c, frontend/plotting/pyplot.c, examples/pyplot_examples/, *features/* | pyplot usability: distinct default names, deck-folder output, linewidth & backend. (1) REAL BUG: in window (interactive) mode pyplot launches the Python viewer in the BACKGROUND (`python3 pyplot.py &`); two plots that both omit the file name shared `pyplot.py/pyplot.data`, and the 2nd call overwrote them before the 1st viewer read them -- both windows showed the 2nd plot (its title and data). Invisible in PNG mode (synchronous). Fix (com_pyplot.c): a per-session counter names successive no-name plots `pyplot`, `pyplot-2`, ... (first unchanged). (2) The .py/.data/.png are written next to the CIRCUIT FILE via ngdirname(ci_filename) rather than ngspice's cwd; the script's loadtxt/savefig carry the deck-dir path so Python finds the data from any cwd; a bare in-folder deck name still lands in the cwd (unchanged). (3) `set pyplot_linewidth=<w>` -> linewidth in the plot()/step() calls. (4) `set pyplot_backend=<name>` -> matplotlib.use() ahead of import pyplot, overriding the auto backend (incl. the Agg forced by the png/svg/pdf terminals). All four opt-in; default output byte-unchanged. filename buffers in pyplot.c grown 128->1024 for full paths. verify_pyplot grows to 13 checks x both solvers; packaged as a standalone feature bundle under features/. Regression 149/149. | | E-184 | frontend/outitf.c, examples/progressbar_examples/* | sweep progress bar reaches 100%. Fix to the E-129 progress bar: it sometimes froze below 100% at end of run (e.g. '[====] 82%' then 'No. of Data Rows'). CAUSE: the 'Reference value' status line is throttled -- redrawn only when >0.25 s has elapsed since the last update -- so the FINAL sweep point's print is almost always skipped (it lands within 0.25 s of the previous tick), freezing the bar at the last throttled value; reaching 100% was never reliable. FIX: outp_print_reference records the latest refval + a bar-shown flag every call; a new outp_finish_reference, called from fileEnd/plotEnd just before the 'No. of Data Rows' line, reprints the bar full at 100% in place using the sweep's TRUE endpoint (TRAN CKTfinalTime, AC ACstopFreq, NOISE NstopFreq, DC last source value). One-shot (reset at analysis start and after firing), silent for non-swept analyses (op/tf), same ft_optimizing/ft_norefprint/cp_background guards -- quiet/background/optimizer runs and the rawfile unaffected. verify_progressbar's end-of-run check tightened from >=90% to ==100% (all four swept analyses); 22/22. Regression 149/149. | | E-188 | spicelib/analysis/dcop.c, include/ngspice/cktdefs.h, frontend/com_sweep.c, examples/warmstart_examples/* | Monte Carlo warm-start (**`montecarlo ... -warm`**). Each MC sample (E-151) re-sources the deck and COLD-solves its DC bias point, running the full gmin/source-stepping homotopy from scratch even though consecutive samples move the operating point only slightly. Profiling: the solve, not the deck re-source, dominates for non-trivial circuits, and every `op` cold-starts -- two IDENTICAL ops back-to-back each burn the full homotopy (~52 Newton iters on a diode ladder; a warm .nodeset at the solution cuts it to ~5). CKTop tries a direct Newton from the caller's firstmode before any homotopy, but DCop always passes MODEINITJCT (cold junction voltages), discarding the previous solution. FIX: `montecarlo -warm` wraps the loop in CKTset-WarmStart(1/0); DCop keeps a buffer (outside the CKTcircuit that reset recreates; indexed by equation number, stable across resets of an identical-topology deck) of the last converged CKTrhsOld, preloads it and calls CKTop with MODEINITFLOAT (use existing node voltages) when a valid same-size guess exists, and snapshots each converged solution. A poor guess just fails the first NIiter and falls through to the normal cold homotopy, so the converged point is identical -- only a cheap failed attempt is wasted. First sample cold, rest warm; when -warm is absent dcop_warm_enable=0 and the path is byte-identical (one extra SMPmatSize read). CORRECTNESS: warm changes only Newton's starting point, so it converges to the same OP to within convergence tolerance -- warm==cold yield EXACTLY at reltol=1e-6, agreeing to within a couple boundary samples at default reltol (v(3)~3.8V, where reltol*|v|~=the narrow spec band; a tolerance effect, removed by tightening reltol). Iteration count ~52->~4 (~13x); wall-clock benefit scales with per-iteration cost (large/hard-converging designs win most; small circuits are dominated by deck re-source + command dispatch). Shared DC-op code, so Sparse+KLU; composes with -lhs. 5 checks (exact at tight reltol, close at default, iteration cut, +lhs, +klu). Regression 152/152. | | E-189 | frontend/com_sweep.c, examples/sweepwave_examples/* | Sweep waveform overlay (**`sweep ... -overlay`**). A usability follow-up to the E-146 `sweep` command: `sweep` steps any knob, runs an inner analysis per point, and records each output's LAST value into a summary `sweep` transfer curve -- but overlaying the full per-point WAVEFORMS to compare shapes meant `setplot`-ing each retained run by hand. `-overlay` (alias `-ov`) collects every point's whole output waveform, resamples them onto a common independent-variable grid, and builds one `sweepwave` plot with a `<output>_<value>` vector per (output, knob value), left current so `plot <out>_<val> ...` works at once -- the HSPICE `.step` overlay in one step. Three static helpers in com_sweep.c: `sw_eval_vec` (evaluate the output expr and copy its FULL waveform + scale into caller buffers; complex/AC stored as magnitude), `sw_interp` (binary-search piecewise-linear resample, flat outside range -- each run lands on its own adaptive time/freq grid, so a shared grid is required before a common scale vector), `sw_wavename` (clean nutmeg name, non-alnum -> _). The sweep loop captures each waveform alongside the unchanged last-value recording; after the loop a common grid over [min,max] of all runs at the finest resolution is built, every waveform resampled onto it. GRACEFUL: a scalar inner analysis (`op`, no waveform) prints "-overlay ignored (analysis '...' has no waveform to overlay)" and yields the ordinary summary -- never an error. Without the flag the path is byte-identical to E-146. Front-end only, solver-independent. Verify: overlaid RC step responses match `1-exp(-t/RC)` (RC=1/2/4us) to worst <1e-4, distinct per-value vectors, graceful op, summary curve intact. 5 checks. Regression 153/153. | | E-190 | frontend/com_sweep.c, examples/nestedsweep_examples/* | Nested multi-knob sweep (**`sweep ... -vs ...`**). Second usability follow-up to the E-146 `sweep` command (after E-189 `-overlay`). `sweep` stepped ONE knob; `-vs <knob> <spec>` (alias `-family`) adds OUTER knobs whose CARTESIAN PRODUCT forms a curve family: the inner (positional) knob stays the x-axis of the summary `sweep` plot, and each combination of the outer knobs yields one curve per output, named `<output>_<outerknob>_<value>...` -- the parametric family of `.dc ... SWEEP ...` / HSPICE nested `.dc`. Up to SW_MAXKNOB=4 knobs, cartesian points capped at SW_MAXPTS. The three-form spec parser (lin/dec/oct, list, start/stop/step) was factored into sw_parse_spec, now shared by the inner knob and each -vs knob so all knobs accept the same forms. SET ORDERING (the subtle part): each knob is applied with its E-146 mechanism -- a .param needs alterparam+reset (re-source), an instance/model knob is in-place alter/altermod -- and a reset drops every in-place alter, so the loop iterates inner-fastest and re-sources ONCE only when a .param knob's value actually changed (outer-loop boundaries; an inner .param still resets every point, a pure in-place sweep never resets), staging all .params via alterparam before the single reset, then RE-APPLYING every in-place knob after the reset (so an inner alter survives an outer .param re-source). By construction a single knob reduces EXACTLY to E-146: one outer combination -> curve named <output>, E-146 messages, identical data indexing (existing sweep/sweepwave examples pass untouched). -overlay composes: each cartesian point's full waveform becomes a sweepwave vector named with EVERY knob value (inner first), so a single knob is <output>_<value> exactly as E-189. New static helpers: sw_parse_spec, sw_set_deck/sw_set_inplace (split from the old sw_set), sw_familyname (summary curve name over outer knobs), sw_pointname (overlay name over all knobs; replaces sw_wavename). Verify: RC low-pass step, R inner x C outer, each family curve == 1-exp(-T/(R*C)) vs R to worst <5e-6, incl. a .param-OUTER knob (reset/alter ordering); cartesian run/curve counts; -overlay family names carry both knob values; single knob still names its curve vo. 5 checks. Regression 154/154. | | E-191 | spicelib/analysis/acan.c, spicelib/analysis/span.c, examples/aclin2_examples/* | **.ac lin 2** / **.sp lin 2** off-by-one fix. Found while auditing the data-output path (wrdata/wrsnp/save/rawfile write): a ragged-length wrdata test wrote an AC vector one point short, which traced not to the writer but to the AC LINEAR frequency sweep. The AC (acan.c) and S-parameter (span.c) analyses computed the linear step behind if (numberSteps - 1 > 1) -- true only for N>=3 -- so exactly lin 2 fell through to the single-point patch (freqDelta = 0). With a zero step the sweep loop (freq += freqDelta; if (freqDelta == 0) goto endsweep;) stops after ONE point, silently dropping the fstop point: a two-point linear AC/SP sweep produced one row. dec/oct (different step formula) and lin 1/3/4/... were all correct; only N=2. The single-point patch (Richard McRoberts) was meant for lin 1; - 1 > 1 over-reached by one. FIX: guard on numberSteps > 1 in both files (the form noisean.c already used correctly), so N=1 -> one point at fstart (patch preserved), N=2 -> freqDelta = fstop-fstart -> two endpoints, N>=3 unchanged. The distortion analysis uses a different N+1-point convention (unaffected); noise was already correct. Verify: RC low-pass, both lin 2 endpoints are GENUINE solved points matching 1/(1+jwRC) to <1e-6 (not a duplicate); lin 1 stays one point; lin N gives N linearly-spaced points (N=3,5,10); .sp lin 2 gives two points with analytic series-R S-params (S11=1/3, S21=2/3). Both solvers. The wider output-path audit found the writers themselves CLEAN: Touchstone wrs2p/wrsnp/rdsnp round-trip across RI/MA/DB, S/Y/Z (Rbase normalization Y*R / Z/R), Hz-GHz incl. complex angles; 2-port special S11 S21 S12 S22 order + N-port row-major both correct for round-trip AND spec interop; wrdata complex (re,im)/singlescale/ragged padding; save filtering restricts the vector set; rawfile write/load bit-exact at 12 digits. 5 checks. Regression 155/155. | | E-192 | frontend/com_checkpoint.c, frontend/com_checkpoint.h, frontend/runcoms.c, examples/autosave_examples/* | Auto-checkpoint on interrupt (**set autosave=<file>**). Follow-up to E-131 checkpoint/restart, prompted by "does savestate fire on Ctrl-C?" -- it did not. E-131 gave savestate/loadstate but a checkpoint had to be taken by hand, so an interrupted long transient lost its progress. E-192: set autosave=<file> makes an interrupted transient auto-write an E-131 checkpoint. SAFE HOOK POINT (the crux): writing a file from a signal handler is undefined behavior, so E-192 does NOT checkpoint in ft_sigintr -- that handler only sets ft_intrpt and returns (during a run ft_setflag is TRUE, so no longjmp). At the next ACCEPTED timepoint, dctran's IFpauseTest (which is OUTstopnow) sees the flag and returns 1, so dctran returns E_PAUSE; if_run maps E_PAUSE to return code 1; dosim (runcoms.c) reports "interrupted" (err equals 1). E-192 hooks exactly that interrupted branch -- on the MAIN thread, at a consistent timestep boundary, with ordinary buffered I/O. CHANGE: com_savestate's body factored into ckt_write_checkpoint(ckt, fname) (the command is now a thin wrapper); the hook calls the SAME function, so an autosave file is byte-for-byte what savestate writes. In the interrupted branch: if cp_getvar("autosave", ...) yields a filename AND the run was a transient (CKTmode and MODETRAN set, CKTtime above 0), call ckt_write_checkpoint. OPT-IN (no var means byte-identical old behavior, nothing written), TRANSIENT-ONLY (the MODETRAN / CKTtime guard blocks stale state after, e.g., an interrupted AC). Interactive-mode only: the SIGINT handler is installed only under non-batch mode, so a batch-mode Ctrl-C still terminates (unchanged). A real Ctrl-C (SIGINT), a `stop when ...` breakpoint, and a Verilog-A stop task all funnel through the same interrupted branch (they differ only in how the flag is set, upstream of E-192). CORRECTNESS: autosave file byte-identical to a manual `savestate` at the same pause; `loadstate` resumes it exactly (E-131 round-trip). Verify (examples/autosave_examples): stop-paused transient writes a checkpoint; equals a manual savestate (byte-identical); loads/resumes; opt-in gate (no var means no file); and a REAL Ctrl-C (SIGINT) via a PTY (interactive) fires it (lenient: skip if the run finishes before the signal, the hook being covered deterministically). E-131 `checkpoint` example still passes (savestate refactor). 5 checks. Regression 156/156. | | E-193 | spicelib/analysis/dcpss.c, examples/pnoiseunits_examples/, *examples/rfpss_examples/verify_rcpnoise.py, examples/rfpss_examples/verify_rccyclo.py, examples/cyclofold_examples/verify examples/pnoisefold_examples/verify_pnoisefold.py, examples/rfpss_examples/rc_pac.cir* | ***`.pnoise`*** *honors* ***`sqrnoise`*** *(noise-density units fix). Found auditing the RF/PSS suite. The ngspice manual (sec. `.noise`, pp. 326-327) defines `onoise_spectrum/inoise_spectrum` as a noise spectral DENSITY in V/sqrt(Hz) (or A/sqrt(Hz)) by default, switchable to the squared V^2/Hz form by the `sqrnoise` control variable. `.noise` obeys this (noisean.c reads `cp_getvar("sqrnoise")` and sqrt's onoise/inoise when unset). But `.pnoise` (periodic noise) ALWAYS emitted the squared V^2/Hz density and IGNORED `sqrnoise` -- so the same-named vectors carried DIFFERENT units across the two analyses (differing by a square), and the documented `sqrnoise` control silently did nothing for pnoise. Confirmed by the toggle: `.noise` flips 4.063e-9 (V/sqrt Hz) <-> 1.651e-17 (V^2/Hz) with `set sqrnoise`; `.pnoise` returned 1.651e-17 both ways. FIX: `pnoise_sweep` (dcpss.c) now reads `sqrnoise` via cp_getvar (added #include cpextern.h) and, when unset (the default), applies sqrt to the emitted onoise/inoise densities -> V/sqrt(Hz); `set sqrnoise` -> V^2/Hz. BOTH emit paths get the conditional sqrt: the E-178 cyclostationary folding branch and the standard adjoint-folding branch. Now `.pnoise` and `.noise` agree by default (match to 0.00). SCOPE: `.pnoise` only. The two-tone `qpnoise` command's single-point diagnostic already prints BOTH forms explicitly (V^2/Hz and V/sqrt(Hz)) so there is no ambiguity, and its swept + single-point paths share the V^2/Hz convention; an initial QPnoiseSweep change was REVERTED to avoid an internal swept-vs-single-point split (verify_qpss_sweep [3] compares them). EXAMPLES: rc_pnoise now checks the default V/sqrt(Hz) form (sqrt of the analytic) and cross-checks vs `.noise` without forcing sqrnoise (match to 0.00); rc_cyclo/cyclofold/pnoisefold are power-density relations (onoisef* == ..., referee sums |Z|^2*S) so they `set sqrnoise` to keep V^2/Hz, and two LTI-limit checks that compared pnoise(V^2) to .noise(V/sqrt Hz)^2 became direct equalities. WIDER AUDIT (all correct): PAC driving-point + the +-1 CONVERSION SIDEBANDS (via the `.pac` card's trailing `maxsideband` arg) match a clean transient DFT to ~0.1%; Harmonic Balance matches a transient DFT to 4-5 digits (an apparent 12x .disto HD3 and 2.4x PAC-sideband discrepancy both dissolved into `fourier`-command interpolation error at high harmonics, resolved by a proper DFT). Also documented the previously-undocumented `.pac` trailing `maxsideband` arg (dcpss.c comment + rc_pac.cir). Verify (examples/pnoiseunits): default pnoise == sqrt(4kTR/(1+(wRC)^2)) (V/sqrt Hz); set sqrnoise == the density (V^2/Hz); default^2 == squared (exact sqrt relation); default pnoise == default .noise (0.00). 4 checks. Regression 157/157.* | | *E-194* | *frontend/com_optimize.c, examples/psoopt_examples/* | Particle swarm optimization (**`optimize -method pso`**). A new search method for the built-in optimizer. E-130/143 gave two LOCAL methods -- Nelder-Mead simplex (`-method nm`, scalar `-minimize`) and Levenberg-Marquardt (`-method lm`, gradient least-squares over `-targets`) -- both of which descend into whichever basin the start point sits in, so on a MULTIMODAL objective they return the local minimum nearest the start. E-194 adds a GLOBAL, population-based, derivative-free method: PSO. N particles fly through the normalized [0,1]^np box; each keeps its own best-seen point (pbest) and the swarm shares a global best (gbest); each iteration updates every velocity `v = chi*(v + phi*r1*(pbest-x) + phi*r2*(gbest-x))` and moves `x = clamp01(x+v)` with the standard Clerc-Kennedy constriction (chi=0.72984, phi=2.05 so the swarm converges without exploding), velocities clamped to half the box, positions to [0,1]. Particle 0 starts at the user init point, the rest uniform-random, so the whole box is explored. Reuses the existing `opt_eval` (returns the scalar cost for both objective kinds), so PSO works for scalar `-minimize` AND `-target` least-squares -- unlike LM, which needs targets. New options: `-swarmsize` <N> (aliases `-swarm`/`-npart`; default auto = 10+4*np capped at 60), `-seed` <s> (PSO draws from a small self-contained splitmix64 PRNG* *independent of ngspice's global RNG, so a seed gives byte-identical results), shared* `-maxiter/-tol` *(tol is the gbest-stagnation tolerance held over several iterations). The nm/lm dispatch is unchanged. Verify (examples/psoopt): on the classic multimodal* `f(p)=sin(p)+sin(10*p/3)` *over [2.7,7.5] (global p*=5.1457 f*=-1.8996, several higher local minima) started at the p=2.7 corner, PSO finds the global (-1.8996, p=5.145) while Nelder-Mead from the same start is trapped at a local minimum (-1.19992, p=3.387); a fixed seed is exactly reproducible; independent seeds all reach the global basin; PSO also solves a `-target` least-squares objective (recovers p to a 1e-12 residual) and a 2-D separable multimodal min (f*=-3.799). Existing optimize example (nm+lm, 20 checks) unchanged. 6 checks. Regression 158/158. | | E-195 | frontend/com_optimize.c, examples/deopt_examples/* | Differential evolution (**`optimize -method de`**). A second GLOBAL method for the built-in optimizer, after PSO (E-194), from the same population-based derivative-free family. DE keeps a population of NP vectors in the normalized [0,1]^np box; where PSO pulls trials toward remembered bests, DE (classic DE/rand/1/bin) builds each trial from a scaled DIFFERENCE of random members and crosses it with the target: `v = a + F*(b-c)` (F=0.8; a,b,c distinct and != i), `u[j] = v[j]` if `rand<CR or j==jrand` else `x[i][j]` (CR=0.9 binomial crossover, one dimension always taken), then greedy selection (keep u in place of x[i] iff cost(u) <= cost(x[i])). The difference vector b-c self-scales to the population's own spread -- large while members are far apart (broad exploration), shrinking as they converge (fine refinement) -- which makes DE robust on rugged / discontinuous / poorly-scaled landscapes with no per-problem step tuning. Reuses the existing opt_eval (scalar cost for both objective kinds), so it works for scalar `-minimize` AND `-target` least-squares like PSO. Shares PSO's infrastructure: the same self-contained splitmix64 PRNG (so `-seed` is byte-reproducible), the `-swarmsize` population option (default auto 10+4*np capped at 60; clamped to >=5 because DE needs 4 distinct members to form a mutant), and shared* `-maxiter/-tol` *(gbest-stagnation). The nm/lm/pso dispatch is unchanged; DE is only selected by* `-method de` *(aliases diffevol, differentialevolution). Verify (examples/deopt): on the multimodal* `f(p)=sin(p)+sin(10*p/3)` *over [2.7,7.5] (global p*=5.1457 f*=-1.8996) started at the trapping p=2.7 corner, DE finds the global (-1.8996, p=5.14574 -- matches p* to 5 digits) while Nelder-Mead from the same start is trapped at a local minimum (-1.19992, p=3.387); a fixed seed is exactly reproducible; independent seeds all reach the global basin; DE also solves a `-target` least-squares objective (recovers p to a 4e-13 residual) and a 2-D separable multimodal min (f*=-3.799); and DE and PSO both reach the global while the local NM does not. Existing optimize (nm+lm, 20 checks) and psoopt (6 checks) examples unchanged. 7 checks. Regression 159/159. | | E-196 | frontend/com_optimize.c, examples/saopt_examples/* | Simulated annealing (**`optimize -method sa`**). A third GLOBAL method for the built-in optimizer, completing the set: after PSO (E-194) and DE (E-195), both population-based, SA is the classic SINGLE-WALKER global optimizer. It holds one current point in the normalized [0,1]^np box; each step it proposes a random neighbour `xn = clamp01(x + step*U(-1,1))` and applies the Metropolis rule -- accept if `cost(xn) <= cost(x)` (downhill), else accept anyway with probability `exp(-(cost(xn)-cost(x))/T)` (uphill). While the temperature T is high uphill moves are readily accepted so the walker climbs OUT of local minima; T is then cooled geometrically (`T *= alpha`, alpha set so T drops ~4 decades over the -maxiter cooling levels, with L=8+4*np moves per level), so uphill moves become rare and it settles into a minimum. The best point ever visited is returned. ONE candidate per step (no population) -> the lightest-weight global method, the natural choice when each analysis is expensive. Everything auto-scales to the problem: T0 = mean |Dcost| of a dozen random probes (so an uphill move of that size is accepted about half the time when hot), and the step size =* `0.30*sqrt(T/T0)+0.02` *shrinks as T cools (broad exploration hot, fine refinement cold) -- nothing to tune. Reuses opt_eval (scalar cost both kinds) -> works for scalar* `-minimize` *AND* `-target` *least-squares. Shares the self-contained splitmix64 PRNG (*`-seed` *byte-reproducible);* `-maxiter` *is the number of cooling levels. The nm/lm/pso/de dispatch is unchanged; SA is only* `-method sa` *(aliases anneal, simulatedannealing). A single walker refines more loosely than a population, so SA reliably reaches the global BASIN rather than machine precision (a short nm/lm polish from its result sharpens it when that matters). Verify (examples/saopt): on the multimodal* `f(p)=sin(p)+sin(10*p/3)` *over [2.7,7.5] (global p*=5.1457 f*=-1.8996) from the trapping p=2.7 corner, SA finds the global basin (f<-1.85, |p-p*|<0.05) while Nelder-Mead is trapped at a local minimum (-1.19992, p=3.387); a fixed seed is reproducible; independent seeds all reach the global basin; SA solves `-target` least-squares and a 2-D multimodal min; and all THREE global methods (sa/pso/de) reach the global while the local nm does not. Existing optimize (20) + psoopt (6) + deopt (7) examples unchanged. 7 checks. Regression 160/160. | | E-197 | frontend/com_optimize.c, examples/opt100_examples/* | A 100-parameter circuit optimization (raised **`optimize`** limits). The built-in optimizer sized its per-run arrays from two compile-time macros; E-197 raises them from OPT_MAXP 16 to 128 (parameters) and OPT_MAXT 64 to 128 (least-squares targets), and lifts the auto-population cap for the PSO/DE population methods from 60 to 256, so a genuinely large problem fits in a single call. Exceeding a cap is reported cleanly (`optimize: too many -param (max 128)`), not crashed. Testbed: 100 independent 1 mA sources, each into an unknown resistor R_i to ground (so v(n_i) = 1e-3 * R_i), fitted to 100 targets t_i = 1e-3 * (100 + 9800*i/99) ohm -- a linear voltage ramp 0.1 .. 9.9 V, a well-posed separable 100-D least-squares. Levenberg-Marquardt is the right tool for a well-posed high-D fit and solves it to machine precision (sum-sq residual 2.5e-29, rms 5e-16, 619 evaluations, ~2 s) with all 100 parameters and 100 targets honored. Two fixes make the derivative-free GLOBAL methods FUNCTION at that scale (they are intrinsically far slower than LM in high dimension, but were failing outright): (1) adaptive DE crossover -- classic DE/rand/1/bin uses CR=0.9, mutating almost every coordinate of a trial; in high dimension a trial that perturbs ~n coordinates at once is essentially always worse than its parent and rejected by greedy selection, so DE freezes at its starting guess. E-197 sets `CR = (n <= 16) ? 0.9 : 15.0/n`, capping the expected mutated-coordinate count at ~15 for large n (exactly the classic 0.9 for n <= 16, so low-D DE is unchanged), and DE descends again -- from ~1240 to ~882 over 35 generations at 100-D. (2) Dimension-scaled convergence patience -- high-D population runs plateau for several generations between improvements, so the gbest-stagnation counter that stops PSO/DE would cut a still-descending run short; the threshold now grows as `8 + n/4` (exactly 8 for n <= 3, so small-n tests are unchanged), giving 33 generations of patience at n=100. DE at 100-D remains a genuinely hard global search that does not fully converge in a short budget -- intrinsic to global optimization in high dimension, not a defect; the message is the division of labor between LM (well-posed high-D fits) and the global methods (multimodal, now usable as dimension grows). Verify (examples/opt100): LM fits all 100 params to 100 targets at machine precision (both raised caps proven at once); DE descends substantially at 100-D (self-calibrated against its own starting cost, >= 15% reduction); a fixed `-seed` is byte-reproducible even at 100 dimensions; and a 129-parameter over-cap run is reported cleanly. Existing optimize (39) + psoopt (6) + deopt (7) + saopt (7) examples unchanged. 4 checks. Regression 161/161. | | E-198 | frontend/com_stb.c, frontend/commands.c, frontend/com_commands.h, frontend/Makefile.am, examples/stb_examples/ | **`stb`** stability / loop-gain analysis. A new front-end command that measures a feedback loop's small-signal loop gain T(f) and reports its phase and gain margin -- the everyday stability check for op-amps, regulators, PLLs -- which ngspice lacked (you had to hand-roll a `.control` script). It is the one-command equivalent of Spectre's `stb` (Middlebrook-Tian double injection). Usage `stb <Vprobe> <Iprobe> <ac-sweep>`: the user marks the loop break with a bias-transparent probe pair between the driving node A and the loaded node B -- a series `Vstb A B dc 0 ac 0` (+node = driver A, -node = load B) and a shunt `Istb 0 B dc 0 ac 0` (ground to load B). Breaking a loop and injecting a single test signal mis-reads the loop gain because of the LOADING at the break (a series voltage injection is exact only from a zero-impedance source into an infinite-impedance load); the double injection cancels that error. `stb` runs two AC sweeps by altering the probe AC magnitudes: voltage injection (Vstb ac=1) gives Tv = -v(A)/v(B); current injection (Istb ac=1) gives the current loop gain from the probe branch current a = i(Vstb) as Ti = -a/(a+1); combined T = (Tv*Ti - 1)/(Tv + Ti + 2). The combined T is INDEPENDENT of where the probe sits in the loop -- the property a single injection lacks -- verified directly (a loaded break gives the same margins as a clean break in the same loop). Numerically, the common clean-break case has a high load impedance so a -> -1 (all injected current returns through the shorted voltage probe) and Ti -> inf, which makes Ti = -a/(a+1) divide by zero; `stb` evaluates the algebraically identical singularity-free form T = -(Tv*a + a + 1)/(Tv*a + Tv + a + 2), exact at a = -1 where it reduces to T = Tv. The complex loop gain is stored as vector `loopgain` vs `frequency` in a new `stb` plot (`plot db(loopgain)` / `plot 180/PI*cph(loopgain)`); phase margin = 180 + angle(T) at the first abs(T)=1 crossing, gain margin = -abs(T)_dB at the first angle(T)=-180 deg crossing (both log-interpolated on an unwrapped phase). Implementation (`com_stb.c`, registered in `commands.c`, declared in `com_commands.h`): recovers the probe's two node names via `CKTinst2Node`, dispatches `alter`/`ac` through the command table (as com_optimize/com_sweep do), and reads the complex vectors v(A)/v(B)/Vstb#branch/frequency with `ft_evaluate`. *Reuses the AC analysis, so both the Sparse and KLU solvers. Verify (examples/stb): PM/GM match the closed-form loop gain of an ideal-output op-amp loop; a loaded break (finite op-amp output impedance) gives the SAME margins as a clean high-impedance break (proving the current injection corrects the loading -- a single voltage injection is ~10% off and diverges past crossover); a 10x-gain design's reduced phase margin is tracked and matches analytics; the complex* `loopgain` *is stored; and an unknown probe is reported cleanly. 5 checks. Regression 162/162. | | E-199 | examples/nport_examples/snp2va.py, examples/nport_examples/* | N-port Touchstone device (snp2va.py). Instantiate a measured or simulated S-parameter block (a filter, cable, connector, package, or amplifier stored in a Touchstone .sNp file) as a circuit element that works in AC AND transient. ngspice had no native n-port element: its .sp "ports" are tagged voltage sources used to MEASURE a circuit's S-parameters, and rdsnp/wrsnp (E-64/E-72) are reader/writer commands, not a device. Rather than a native convolution device (with its own history buffers and stability headaches), snp2va.py converts the file into a Verilog-A n-port realized with laplace_nd, so OpenVAF's OSDI laplace machinery (E-31 complex poles) supplies BOTH the AC response and the transient (recursive-state) response with no convolution code. Pipeline: parse Touchstone (any port count; S/Y/Z data; MA/DB/RI formats; Hz..GHz; arbitrary reference impedance) -> S(f) -> Y(f)=(1/z0)(I-S)(I+S)^-1 -> common-pole vector fit (Gustavsen) -> emit I(p_i) <+ sum_j [laplace_nd(V(p_j), num_ij, den) + e_ij*ddt(V(p_j))]. Two numerical details are essential. (1) The vector fit MUST be done in NORMALIZED frequency -- without it the s*e column is ~1e9 while the 1/(s-p) columns are ~1e-7, so the least-squares is hopelessly ill-conditioned and the poles never relocate (the pole relocation is done as polynomial roots of the sigma numerator via Durand-Kerner, not a matrix eigensolve). (2) Y is IMPROPER whenever a network has shunt capacitance (Y ~ sC at high frequency), which laplace_nd -- a ratio of polynomials -- cannot represent (it gave a 63 percent AC error at the band edge); the fix is to give laplace_nd only the strictly-proper (pole) part and emit the s*e *term as an explicit ddt (a capacitance), dropping the AC error to ~1e-6. HARDENING: automatic order selection CLIMBS the pole count to capture high-order and delay-dominated responses (a pure delay is uniformly poor at low orders, so the selection must not quit early on a small between-order improvement), but stops at the KNEE once the error has already come down near a floor -- past that knee extra poles just fit measurement noise into an over-fitted, ill-conditioned, unstable model, so stopping there fits clean data to machine precision and noisy data at its noise floor. It breaks when a fit turns unstable (polynomial-root finding degrades at high degree) and returns the best STABLE fit. Stability is enforced (right-half-plane poles reflected into the left half plane) so the emitted model is always BIBO-stable even for a noisy non-passive fit; passivity is checked and reported (enforcement by residue perturbation is a documented non-goal for now). Pure Python standard library, NO numpy: least-squares via Householder QR, complex matrix inverse via Gauss-Jordan, and polynomial roots via Durand-Kerner are all reimplemented (each validated against numpy to machine precision during development). Verify (examples/nport): each check generates a Touchstone file from a network whose response is known exactly, runs snp2va.py + OpenVAF, and confirms the device matches the ORIGINAL network in ngspice -- an R-L-C resonator to 5e-6 in AC (including the transmission peak) and 5e-3 in transient (one model, both analyses); a 5-pole LC ladder (order selection scales past 2 poles); a 3-port star network to 4e-9 on both coupled outputs; and a noisy measurement fit at its noise floor, reported non-passive, staying bounded in transient. Stress-tested to near machine precision on resonators, notches, high-Q blocks, and multi-pole ladders; a delay-dominated transmission line degrades gracefully (AC accurate over the fitted band, transient pulse edges ring -- inherent to rational fitting of a pure delay, for which a T/LTRA line is the right model). 6 checks. Regression 163/163. | | E-200 | src/frontend/snp2va.c, src/frontend/snp2va.h, src/frontend/com_presnp.c, src/frontend/commands.c, src/frontend/inp.c, src/frontend/inpcom.c, src/frontend/Makefile.am, examples/presnp_examples/* | Built-in pre_snp command. Folds the E-199 Touchstone-to-Verilog-A converter into ngspice itself as a C command, so the workflow no longer needs an external Python script: inside a .control block, pre_snp <file.sNp> [module] parses the .sNp, common-pole vector-fits it, writes the .va, and invokes openvaf-r to compile the .osdi -- a one-line analog of pre_osdi. snp2va.c is a faithful C port of snp2va.py with identical numerics: it uses C99 double *Complex (typedef'd cplx, so it does not collide with ngspice's own complex.h macros) and reimplements the three vector-fitting primitives -- least-squares via Householder QR, complex matrix inverse via Gauss-Jordan, polynomial roots via Durand-Kerner -- plus the same parse -> S(f) -> Y(f)=(1/z0)(I-S)(I+S)^-1 -> vector fit -> emit I(p_i) <+ sum_j [laplace_nd(V(p_j),num,den) +*

*e_ij*ddt(V(p_j))] pipeline, the same normalized-frequency fit, the improper e*s (shunt-C) term split out as an explicit ddt, and the same automatic order selection returning the best STABLE fit (RHP poles reflected -> BIBO-stable). It was checked numerically identical to the Python reference (resonator RMS 4e-6, ladder 7e-7, 3-port 3e-8). The public entry point snp2va_convert(snpfile, vafile, module, msg, msglen) is called by the command wrapper com_pre_snp (com_presnp.c), which then shells out to openvaf-r. ORDERING: the command is registered under the name snp, so writing pre_snp in a deck triggers ngspice's existing pre* mechanism (inp.c strips the pre_ prefix and runs the command before the circuit is parsed). That alone is not enough -- pre_snp must run before the pre_osdi that loads its output, and control lines otherwise execute in deck order -- so the collected pre-commands are now executed in TWO PASSES: pass 0 runs every snp command (in deck order), pass 1 runs the rest. The .osdi a pre_snp generates is therefore guaranteed to exist by the time any pre_osdi loads it, EVEN IF the deck lists the pre_osdi line first; the ordering is a guarantee, not a convention the user must remember. snp/pre_snp are also added to inpcom.c's case-preservation list (beside osdi/pre_osdi) so the file-path argument keeps its original case. The compiler is located by find_openvaf(): the openvaf ngspice variable (set openvaf=... in spinit or on the command line -- a set inside .control runs too late, after the pre-commands), then the OPENVAF environment variable, then $SPICE_LIB_DIR/openvaf-r, then PATH; if none resolve, pre_snp reports the failing openvaf-r invocation and lists all four options. Verify (examples/presnp): each check builds a Touchstone file from a network whose response is known exactly, lets pre_snp do the whole convert+compile+load inside ngspice, and confirms the device matches the ORIGINAL network -- pre_snp writes the .va and compiles the .osdi; the device matches an R-L-C resonator to 5e-6 in AC (including the transmission peak) and to 5e-3 in transient (one compiled block, both analyses); a 3-port star network compiles and matches on both coupled outputs to 4e-9; and with pre_osdi written BEFORE pre_snp the model still loads, proving the ordering guarantee. 5 checks. Regression 164/164. HARDENING (from an N-port stress sweep -- generate an N-port RLC network, extract S-params with .sp, round-trip through pre_snp, compare DC/AC/transient vs the original subcircuit): two defects the 2-pole checks never reached, both fixed in snp2va.c. (1) Order-selection double-free: the climb that keeps the best STABLE fit let its best and prev buffers alias and then freed the same allocation twice, so pre_snp aborted (SIGABRT) on any network whose fit needed >=3 pole orders (the 2-pole resonator/star escaped by breaking at tolerance first); fixed by not freeing a buffer the other pointer still holds. (2) Transient divergence from a non-passive realization: each Y_ij is fit independently, so the model could blow up in transient (v -> ~1e284) even though every pole is in the LHP and AC is exact -- a single degree-Np rational per element has coefficients spanning ~abs(p)^Np (~1e79 for 8 poles at 1e10 rad/s, an ill-conditioned companion form), and the improper e*s terms form a capacitance matrix with no passivity constraint (an indefinite "negative-capacitance" matrix -> unstable DAE); fixed by realizing each element as a PARALLEL bank of first/second-order laplace_nd sections (every coefficient <= O(abs(p)^2)) and projecting the e-matrix onto the symmetric PSD cone (symmetrize -> Jacobi eigendecompose -> clamp negative eigenvalues to 0). The presnp example gained a 4-port coupled-ladder check (fit climbs past two orders; transient must stay bounded and match) that fails on the pre-fix converter. Regression 164/164.* | | E-201 | *src/frontend/snp2va.c, examples/presnp_examples/* | pre_snp scalability (fast vector fitting + shared-pole realization). The E-200 converter struggled from about 8 ports up (an 8-port took ~190s and larger port counts were infeasible). Two things scaled badly, both in snp2va.c: the pole identification stacked all N*N elements into one dense least-squares of size (2*NsN*N) x (N*N(Np+2)+Np) -- O(N^4) memory, O(N^6) compute, re-solved dozens of times over the order climb (~8TB of RAM at N=100); and the emitted model gave every one of the N*N elements its own laplace_nd bank -- O(N^2*Np) filter sections and OSDI state. Four fixes rebuild the scalability. (1) FAST VECTOR FITTING (Deschrijver/Gustavsen): the stacked pole-ID matrix has block-arrow structure -- each element's residue columns are independent and couple only through the shared common-pole (sigma) columns -- so instead of forming that matrix, each element block is Householder-reduced on its own (a small 2Ns x (Np+2) QR) and only its residual rows (orthogonal to that element's own columns) are stacked into one tall ((2Ns-Np-2)*Ne) x Np system solved for sigma; memory O(N^4)->O(N^2), compute O(N^6)->O(N^2*Ns*Np^2). (2) RECIPROCITY: a passive network has symmetric Y, so when detected only the upper triangle (N(N+1)/2 elements) is fit and residues are mirrored (~2x fewer solves). (3) ITERATION CAP: pole relocation stops once the poles settle (relative move < 1e-4) instead of a fixed 12 sweeps (typically 3-5). (4) SHARED-POLE REALIZATION: all N*N elements share the same poles, so the pole-filters are computed ONCE per input port -- a real pole gives one filter V(pj)/(s-p), a conjugate pair two real basis filters (s-sigma)/D and omega/D, with the residue entering each output as a real scalar weight -- and each output current is a cheap weighted sum; this is O(N*Np) laplace_nd and OSDI state instead of O(N^2*Np), the difference between a model that compiles/simulates at large N and one that does not. Three latent bugs the stress test also surfaced, all fixed: (a) an order-selection double-free -- at higher pole counts a relocated pole set could lose its adjacent-conjugate-pair structure and build_basis/ctil_to_cres (which walk i+=2 on a complex pole) wrote one slot past the residue array; fixed by a canon_poles step that snaps near-real poles to real and rebuilds exact adjacent conjugate pairs. (b) a 256-entry stack array -- the Touchstone parser assembled each frequency's matrix in a fixed cplx pv[16*16], hard-capping the port count at 16 (N=32 smashed the stack); moved to the heap, auto-detect limit lifted to 512. (c) an OpenVAF crash -- a laplace_nd coefficient array assigned to a variable (as the shared realization does) crashes OpenVAF when a coefficient is an integer literal like '{1}' (which %.12g prints for 1.0); the emitter now forces a real literal '{1.0}'. Results: fit at N=8 220s -> 3.6s; N=64 fits in 47s (was ~860GB of RAM); N=100 fits in 2.75min at 9.6e-4 error; model laplace_nd count now N*Np (65 at N=8, 449 at N=32) not ~N^2*Np/2 (256, 7168). Correctness preserved: DC/AC/transient still overlay the original subcircuit (N=8 unchanged; N=16, previously infeasible, matches to 1.3%). The remaining bottleneck is OpenVAF's compile time for the (now far smaller) shared model (~26s at N=16, ~43s at N=20), so the practical end-to-end ceiling is ~N=20-24 while the fit itself scales to N=100. Verify (examples/presnp): an 8-port coupled ladder converts+compiles in a few seconds with the compact shared model (65 laplace_nd for 8 ports) and matches the original network in AC; the E-200 order/realization guards and resonator/3-port checks still pass. 9 checks. Regression 164/164. |

(E-25's simparam exposure lives in the OSDI callback table populated at load time; its diff rides inside the `osdiload.c`/callbacks changes.)

Build-warning cleanup (post-E-127): a full rebuild under the repo's `-Wconversion` flags surfaced latent warnings. `configure.ac` had `-s` (a strip flag) in the compile `CFLAGS`, so clang warned "argument unused during compilation: '-s'" on every source file (~1526 warnings) — removed (binaries are stripped separately by the fold and CI). `-Wconversion` also flagged a real latent bug in `maths/ni/nipzmeth.c` (`b = 0.0` was an assignment where `b == 0.0` was intended) — fixed — and benign `double`→`bool` conversions in `maths/cmaths/cmath4.c` — made explicit. Full-build warnings dropped 1903 → 394; the remainder are `-Wconversion` warnings in third-party SuiteSparse (KLU) and CMC device models (HiSIM), which are left to their upstream sources.

Every entry above is guarded by a verify script under `examples/` and was regression-checked against the full example arsenal, the compiler's integration suite, and the VA_TEST industry-model corpus at the time it landed.